computation

MDPI

Article

# Performance-Enhancing Market Risk Calculation Through Gaussian Process Regression and Multi-Fidelity Modeling

**N. Lehdili [1,*], P. Oswald [1] and H. D. Nguyen [2]**

[1] Market and Counterparty Risk Modeling (MCRM), Enterprise Risk Management Department (ERM), Natixis CIB, 75013 Paris, France; pascal.oswald@natixis.com
[2] Laboratoire de Probabilités, Statistique et Modélisation, Université Paris Citeé, 75006 Paris, France; hdnguyen@lpsm.paris
* Correspondence: noureddine.lehdili@natixis.com

**Abstract:** The market risk measurement of a trading portfolio in banks, specifically the practical implementation of the value-at-risk (VaR) and expected shortfall (ES) models, involves intensive recalls of the pricing engine. Machine learning algorithms may offer a solution to this challenge. In this study, we investigate the application of the Gaussian process (GP) regression and multi-fidelity modeling technique as approximation for the pricing engine. More precisely, multi-fidelity modeling combines models of different fidelity levels, defined as the degree of detail and precision offered by a predictive model or simulation, to achieve rapid yet precise prediction. We use the regression models to predict the prices of mono- and multi-asset equity option portfolios. In our numerical experiments, conducted with data limitation, we observe that both the standard GP model and multi-fidelity GP model outperform both the traditional approaches used in banks and the well-known neural network model in term of pricing accuracy as well as risk calculation efficiency.

**Keywords:** value-at-risk; expected shortfall; Basel III; FRTB; option pricing; multi-asset option; Gaussian process regression; multi-fidelity model; machine learning; neural networks

**MSC:** 91G20; 91B05; 62G08; 60G15; 65D05

check for updates

Academic Editors: Jerzy Rosłon, Michał Podolski and Bartłomiej Sroka

Received: 3 February 2025
Revised: 6 May 2025
Accepted: 8 May 2025
Published: 3 June 2025

**Citation:** Lehdili, N.; Oswald, P.; Nguyen, H.D. Performance-Enhancing Market Risk Calculation Through Gaussian Process Regression and Multi-Fidelity Modeling. *Computation* **2025**, *13*, 134. https://doi.org/10.3390/computation13060134

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

## 1. Introduction

Market risk refers to the potential for losses resulting from adverse price movements in financial markets. It affects both on- and off-balance sheet positions of financial institutions. This risk mainly stems from trading activities and from exposures—among others—to fluctuations in equities, indices, interest rates, commodities, and exchange rates across the bank's entire balance sheet. Since the 2008 subprime crisis and the Greek crisis, banking regulations have been reinforced to ensure adequate capital reserves based on the risks banks take. The capital estimation relies on risk metrics such as value-at-risk (VaR) and expected shortfall (ES) [1–3]. As a result, the required computational capacity has increased significantly—banks estimate that the number of full revaluation runs will grow by a factor of 5 to 10 [4] compared to current VaR calculations. In response to these challenges, the banking industry is increasingly exploring the use of machine learning [5–7]. This technology is increasingly valuable for managing large data sets, improving efficiency in areas such as pricing, fraud detection, and portfolio management, and optimizing processes to meet FRTB requirements while streamlining banking operations. Research on market risk modeling has intensified the use of machine learning algorithms in financial risk

management, although their deployment remains limited in classical quantitative finance. Models like artificial neural networks (ANNs) are used for options pricing and volatility forecasting, thus reducing computation times. More recent approaches employ Gaussian processes and Bayesian optimization algorithms to accelerate derivatives pricing, value-at-risk calculation, and expected shortfall. Despite these advances, no single machine learning method has yet dominated financial risk measurement. However, these techniques hold promise for enhancing the efficiency of regulatory calculations, particularly in the context of the FRTB requirements [4,8].

A parallel line of research investigates the use of Gaussian processes (GPs) and Bayesian optimization in finance [9–13]. The GP is a generalization of a Gaussian random vector and can model stochastic processes on continuous functions, where the covariance matrix is replaced by a kernel function. Two notable applications of Gaussian process regression (GPR) have recently been published. The first focuses on fast derivative pricing and hedging, particularly in markets modeled with a limited set of parameters [10]. This approach is efficient for statistical learning, as the small number of input parameters allows the machine to quickly learn the function. The main advantage, as highlighted in [10], is the significant reduction in computation time while accurately learning exotic option prices, sensitivities, and hedge parameters. The second contribution [13,14] addresses calculating value-at-risk (VaR) and expected shortfall for a portfolio of derivatives, where the revaluation of the portfolio is performed as a linear combination of instrument prices using the GPR method. Numerical tests show that the GPR approach yields identical results to full revaluation and outperforms the Taylor expansion method. These results suggest promising applications in risk management and regulatory capital calculations. We aim to further explore the quantitative analysis and potential improvements in pricing, as well as VaR and expected shortfall computations for an entire trading book.

### *1.1. Research Focus and Contributions*

This paper proposes a new approach to market risk measurement by extending Gaussian process regression (GPR) to high-dimensional, multi-asset derivatives portfolios, through the use of multi-fidelity modeling. To our knowledge, this is the first application of multi-fidelity Gaussian process regression (mGPR) in the field of quantitative finance. In doing so, we address two major challenges currently faced by financial institutions: the curse of dimensionality in machine learning models applied to derivatives portfolios and the computational burden of daily revaluations under FRTB constraints. Unlike previous applications of GPR in financial risk, which focused on pricing single instruments or low-dimensional portfolios, our method directly learns portfolio-level risk metrics in a high-dimensional setting by combining low- and high-fidelity approximations. To compute risk metrics such as VaR and ES for large derivative portfolios, banks typically rely on Monte Carlo simulations and complex pricing libraries—a process that remains computationally intensive and complex, especially under recent regulatory requirements [4,8]. Beyond regulatory compliance, banks face growing pressure to optimize the cost and speed of market risk calculations. In practice, the number of risk factors in large trading books and the complexity of pricing models make traditional full revaluation approaches computationally unfeasible on a daily basis. Thus, there is a strong incentive to develop surrogate models capable of providing accurate estimations of portfolio risk metrics in reduced time, without sacrificing precision or regulatory credibility. Our work is motivated by real implementation challenges encountered in market risk desks, where computing VaR/ES for multi-asset portfolios under FRTB requirements [4,8] may require hours of CPU time per portfolio. We show that our mGPR-based approach can dramatically reduce that burden. In our previous work [13], we applied GPR to estimate the market risk of

single-asset derivatives portfolios by training the model on entire portfolio values rather than individual instruments [15]. This allowed significant computational gains while preserving accuracy. However, as the dimensionality increases—particularly with multi-asset options such as best-of, worst-of, and basket options—the curse of dimensionality becomes a major obstacle, making traditional GPR approaches impractical. To overcome this, we propose using multi-fidelity Gaussian process regression (mGPR), a technique that combines high-fidelity models (e.g., full pricing engines) with low-fidelity approximations (e.g., faster but less precise pricing schemes), as suggested in recent advances in surrogate modeling [16]. We implement a parallel training process based on selected risk factors to manage calibration with limited data sets. This allows efficient and scalable VaR and ES computations in the context of complex, high-dimensional portfolios. In summary, this paper contributes to the literature by (i) extending the use of GPR to multi-asset portfolios through a multi-fidelity framework, and (ii) addressing the dual challenge of computational efficiency and modeling accuracy in market risk management for complex derivatives:

1. *Extension of Gaussian Process Models to Address the Curse of Dimensionality in Multi-Asset Option Portfolio Valuation:* A key contribution of this paper lies in extending Gaussian process (GP) regression models to address the curse of dimensionality in multi-asset derivative pricing [17]. This phenomenon, which causes the number of data required for training machine learning models to increase exponentially as the input space (number of risk factors or assets) grows, significantly impacts the efficiency of models, especially in the context of multi-asset options. While GP models have shown effectiveness in pricing financial instruments in low-dimensional settings, their scalability is difficult when applied to multi-asset derivatives. This paper identifies the current limitations in the application of GP models, such as in the pricing of fixed-income derivatives (e.g., Bermudan swaptions) [18], where the high-dimensional data required for training leads to significant computational challenges. This paper proposes improvements to GP techniques to mitigate the dimensionality problem while preserving the predictive power of the models. This extension represents a significant advancement in the application of machine learning techniques, particularly GP models, to more complex financial scenarios involving many assets or risk factors.
2. *Introduction of Multi-Fidelity Modeling in Quantitative Finance:* The second key contribution of this paper is the introduction of multi-fidelity modeling into quantitative finance [19], an innovative approach that combines models of varying accuracy to enhance the efficiency and accuracy of risk calculations. Multi-fidelity modeling leverages the strengths of high-fidelity models (which provide highly accurate results but are computationally expensive) and low-fidelity models (which offer faster results but with some loss of accuracy). In the context of financial derivatives pricing, high-fidelity models often represent detailed and computationally expensive pricing engines, while low-fidelity models can be simpler approximations or faster pricing engines for related products. By integrating these models, multi-fidelity modeling allows more efficient risk management and faster derivatives pricing, particularly for complex instruments involving many risk factors [12]. This paper specifically explores the use of multi-fidelity Gaussian process regression (mGPR), a technique that combines the power of GP models with the principles of multi-fidelity modeling. While mGPR has been widely used in fields such as geostatistics and physics (under the name cokriging) [20], this paper applies it to quantitative finance for the first time, offering a promising new approach to improving the speed and accuracy of financial predictions. This contribution opens new avenues for practical applications in risk modeling, where computational resources are often a limiting factor.

### 1.2. Structure of the Paper

The structure of this paper is organized as follows. Section 2 introduces the principle of market risk according to the regulation, highlighting the computational challenge in the practical implementation of the VaR and ES model. Section 3 reviews the Gaussian process regression and its application in the risk calculation of the trading portfolios. Section 4 describes multi-fidelity modeling and how it works with the GP model, including some ideas of their application in quantitative finance. Section 5 sketches the setup and the training specification for our numerical experiments, which are reported in Section 6. Section 7 concludes our findings and provides perspectives for future research.

## 2. Market Risk Assessment and Computational Challenge

According to Basel Committee on Banking Supervision [4], market risk is defined as *"the risk of losses (in on- and off-balance sheet positions) arising from movements in market prices"*. Market risks include default risk, interest rate risk, credit spread risk, equity risk, foreign exchange (FX) risk commodities risk, and so on. Regulation separates market risk with other kinds of financial risk, for example, credit risks referring to the risk of loss resulting from counterparty (the party with whom the bank makes engagement in the contract) default or operational risk referring the risk of loss resulting from the failures of internal banking system. Under the FRTB framework, banks can choose either the standardized measurement method (SMM) or the internal model-based approach (IMA) to quantify their market risks. The first approach, which can be easily implemented, is not appealing to banks due to its overestimation of the capital required to cover market risks. The second approach more accurately reflects the risk sensitivities and better postulates the economic risk carried by the banking balance sheet. However, to use their own proposed internal model, banks must gain the regulatory approval through a rigorous backtesting procedure. Under the internal model approach, the calculation of capital charge is based on risk metrics, namely, value-at-risk (VaR) and expected shortfall (ES), of the possible loss of the trading portfolio during a certain trading period. VaR and ES are measured at typically high confidence levels $\alpha = 97.5\%$ (for ES) or 99% (for VaR) in a short liquidity horizon, i.e., $h = 1$ day or 10 days. Assume that the value of a considered trading portfolio $V_t$ at date $t$ is a function of risk factors $RF_t$, denoted by $p_t(RF_t)$. At date $t + h$, the possible loss of the portfolio (or profit and loss), denoted by $L_t(h)$, is defined by

$$L_t(h) = -(V_{t+h} - V_t) = p_t(RF_t) - p_{t+h}(RF_{t+h}) \approx p_t(RF_t) - p_t(RF_t + \Delta RF_h), \tag{1}$$

where $V_{t+h} = p_{t+h}(RF_{t+h})$ is the value of portfolio at date $t + h$ . The value-at-risk at a confidence level $\alpha$ represents the minimum capital required to cover the market risk, while the expected shortfall means the average loss given the latter exceeds the VaR. These two risk measures are defined by

$$\begin{aligned} \mathrm{VaR}(L_t, h, \alpha) &= \min\{q \in \mathbb{R} : \mathrm{P}[L_t(h) \leq q] = \alpha\}, \\ \mathrm{ES}(L_t, h, \alpha) &= \mathrm{E}[L_t(h) | L_t(h) \geq \mathrm{VaR}(L_t, h, \alpha)] = \frac{1}{1-\alpha} \int_\alpha^1 \mathrm{VaR}(L_t, h, \gamma) d\gamma. \end{aligned} \tag{2}$$

Statistical methods for calculating (2) include analytical, historical, and Monte Carlo approaches ([21], Section 2.2, p. 61). The Monte Carlo approach is the one currently used by most banks due to its accuracy and ability to model complex risk factors. In this method, $\mathrm{VaR}(L_t, h, \alpha)$ is the $\alpha$-percentile of the set of possible losses simulated for time horizon $h$, while $\mathrm{ES}(L_t, h, \alpha)$ is the empirical average of losses above the $\mathrm{VaR}(L_t, h, \alpha)$. We highlight that the market risk factor shocks at time $t + h$, which may lead to potential losses, are generated using synthetic diffusion models. These models must first be calibrated with

real historical market data. Once calibrated, the Monte Carlo-based market risk calculation process is outlined in Algorithm 1 (see also [22] for a review of Monte Carlo risk assessment).

**Algorithm 1.** Calculation of VaR and ES by full repricing approach.

**input :** Calibrated diffusion model, pricing engine $p_t$, an actual vector of risk factors $RF_t$, a confidence level $\alpha$, time step $h$, and a large number $M$

**output:** estimated $\mathrm{VaR}(L_t, h, \alpha)$ and $\mathrm{ES}(L_t, h, \alpha)$

1 Compute $p_t(RF_t)$
2 Simulate $M$ scenarios of shock $\Delta RF_h$ using calibrated diffusion model
3 Compute $M$ corresponding prices $p_t(RF_t + \Delta RF_h)$
4 Compute $M$ scenarios of losses $L_t(h) = -(p_t(RF_t + \Delta RF_h) - p_t(RF_t))$
5 Compute VaR and ES by Monte Carlo

### *2.1. Computational Challenge and Applications of Machine Learning*

The market risk calculation appears to be achievable by Algorithm 1. Nevertheless, we have not yet discussed the computational complexity of the pricing engine. In banks, the pricing engine of a trading derivative portfolio involves various numerical algorithms depending on the nature of the portfolio. Some products or portfolios can be rapidly priced, such as using analytical formulas or accurately fast approximations. However, when financial models and/or products of the portfolio are complex, as is usually the case in practice, some heavily computational algorithms need to be referenced, including Monte Carlo schemes, tree methods ([23], Chapters 6 and 7), and least square methods [24]. Risk calculation using any of the above algorithms is referred to as the full repricing approach (i.e., full revaluation approach). While highly accurate, it is computationally expensive—especially for a portfolio of complex products—as computing the tail risk measures like VaR and ES requires intensive calls of the pricing engine.

In the context of implementing the VaR and ES calculation using Monte Carlo simulation, banking institutions tend to avoid direct use of pricers, as it is time consuming and makes them inefficient. Certain banks prefer to use proxies based on sensitivity calculation and second-order Taylor developments to compute the possible losses of derivative portfolios [1], see Appendix C. Although these approaches offer performance gains, their precision is often questioned. Recently, alternative approaches have emerged that aim to strike a balance between computational performance and precision in risk calculation. Among these methods, the works of [9,10,25] explore the application of Gaussian process regression in finance, and [15] discuss the use of neural networks in the same context. The common idea behind these studies is to use machine learning models as interpolation tools, also known as surrogate models, to estimate derivative prices. More recently, ref. [26] used Chebyshev polynomials as interpolators, combined with principal component analysis techniques (PCA), for financial risk calculation, aligning with the same line of thought. Figure 1 illustrates the risk calculation process that employs these innovative machine learning approaches, involving only a few complete revaluations of a derivative portfolio.

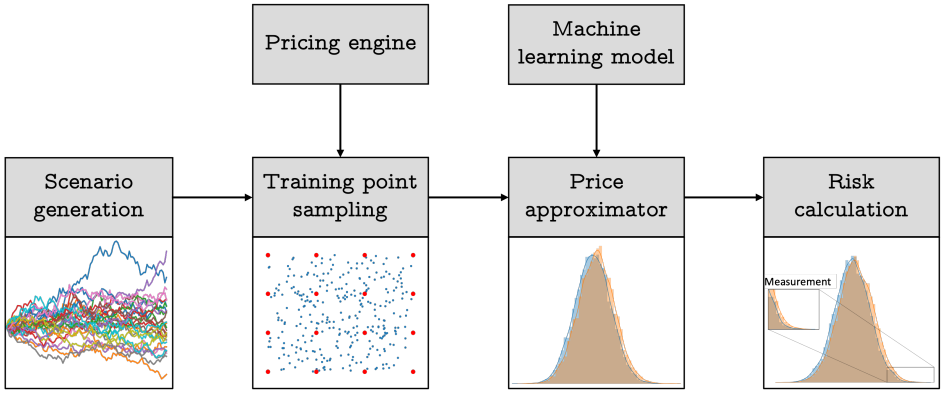


**Figure 1.** New Monte Carlo risk calculation procedure using machine learning for approximating the pricing engine.

### 2.2. Equity Options

In the remainder of this paper, we focus on evaluating and managing the market risk of an equity derivatives portfolio, though our numerical approaches apply to other portfolios as well. These products are written on a stock whose price at date $t$ is denoted by $S_t$. Examples include vanilla options, barrier options, and American options on a single underlying asset. In the Black–Scholes model, the values of these options are simply determined by analytical formulas depending on the underlying asset price $S_t$, except the American option values for which a numerical method, e.g., binomial tree, is mandatory. The portfolio may also include multi-underlying options such as basket options, best-of options, and worst-of options. In this case, we denote by $S_t = [S_t^1, \dots S_t^d]$ the prices of $d$ underlying assets that are governed by lognormal processes:

$$\begin{aligned} dS_t^j &= rS_t^j dt + \sigma^j S_t^j dW_t^j, \quad \text{for } j = 1 \dots d, \\ d\langle W^j, W^{j'} \rangle_t &= \rho_{jj'} dt, \end{aligned} \tag{3}$$

where $r$ is a risk-free rate, $\sigma^j$ denote the asset volatilities, and $W^j$ are Brownian motions with a correlation matrix $\mathbf{R} = (\rho_{jj'})_{j,j' \in \{1,\dots,d\}}$. In the following, we assume the constant risk-free rate and volatilities.

Let $\eta(S_T)$ denote the value of the European multi-asset derivative at the maturity $T$, also called the payoff. Subject to the above assumptions, the value of the European multi-asset option $V_t$ at date $t$ is the conditional expectation under the risk neutral probability of discounted payoff, denoted by $\mathrm{E}^*$:

$$p_t(S_t) = \mathrm{E}^*[e^{-r(T-t)}\eta(S_T)|S_t].$$

We are particularly interested in four options whose payoff can be written as a vanilla option payoff against a strike $K$ given in Table 1.

**Table 1.** Multi-asset options covered in our numerics.

| Option | Geometric Average Call/Put | Basket Call/Put | Best-of Call/Put | Worst-of Call/Put |
|---|---|---|---|---|
| Payoff | $\left(\left(\prod_{j=1}^{d} S_T^j\right)^{\frac{1}{d}} - K\right)^{\pm}$ | $\left(\sum_{j=1}^{d} \alpha_j S_T^j - K\right)^{\pm}$ | $\left(\max_j S_T^j - K\right)^{\pm}$ | $\left(\min_j S_T^j - K\right)^{\pm}$ |

For basket options, $\sum_{i=1}^{d} \alpha_j = 1$. The case $\alpha_j = \frac{1}{d}$, for $j = 1, \dots, d$, defines arithmetic average options.

Given (3), the price of geometric average options is quickly referenced by the Black–Scholes formula, and the price of basket options can be accurately approximated by the Black–Scholes formula due to the convention that the sum of lognormal variables is approximately a lognormal variable [27]. Otherwise, the price of other options in Table 1 must be evaluated by Monte Carlo simulation.

**Remark 1.**

***(i)*** *Before applying VaR and ES calculations (Algorithm 1), banks must identify relevant market risk factors, define a diffusion model, and calibrate its parameters using historical data. In this paper, we assume that these steps have already been completed and focus on improving the risk calculation method.*

***(ii)*** *With a large number of Monte Carlo paths, pricing functions become smooth regardless of diffusion model complexity—an essential property for regression-based methods. We use a log-normal diffusion model for its balance between realism and analytical simplicity.*

***(iii)*** *Market risk factors often involve term structures like yield curves or volatility surfaces. Modeling shocks to all such factors increases dimensionality and complexity. A common simplification is to focus only on diffusive factors (e.g., S), as in the composition technique. Throughout this paper, "market risk factors" refer specifically to these diffusive components.*

## 3. Gaussian Process Regression for Option Pricing

Gaussian process regression is the canonical method for Bayesian modeling of spatial functions. The method is highly recommended in cases where data are sparse because of its generalization of the Gaussian distribution from finite-dimensional vector spaces to infinite-dimensional functional spaces. Due to this principle, the model is considered as nonparametric. In this section, we briefly introduce the Gaussian process regression and its application to model financial derivative products. The background of the GP model can be found in [28] and ([29], Chapter 15).

In the following, bold lowercase letters, e.g., $\mathbf{x}$, stand for vector notation, whereas bold uppercase letters, e.g., $\mathbf{X}$, are used for matrix notation.

### *3.1. Gaussian Processes Regression and Prediction*

Let $(\Omega, \mathcal{A}, \mathrm{P})$ be a probability space, and consider a random vector $\mathbf{x} : \Omega \to \mathcal{X} \subset \mathbb{R}^d$ for a positive integer $d$ corresponding to the dimension of inputs, and a response variable $y \in \mathbb{R}$. In supervised learning, the goal is to learn a target function $f : \mathcal{X} \to \mathbb{R}$ that portrays the relation between inputs and the response variable. To this end, we assume the following regression:

$$y_i = f(\mathbf{x}_i) + \epsilon_i, \tag{4}$$

where $\epsilon_i$ are i.i.d. Gaussian white noise with zero mean and variance $\sigma^2$. Unlike other regression models which usually assume a parameterization form of $f$, the Gaussian processes directly place a multivariate normal prior in the space of functions. Implicitly, for any set of input points $[\mathbf{x}_1, \ldots, \mathbf{x}_N]$ in $\mathcal{X}$, the random vector $[f(\mathbf{x}_1), \ldots, f(\mathbf{x}_N)]^\top$ is multivariate normally distributed. More precisely, given a data set of $N$ observations $\mathcal{D} = (\mathbf{X}, \mathbf{y}) = \{(\mathbf{x}_i, y_i) \mid i = 1, \ldots, N\}$, where $\mathbf{x}_i$ is taken from set $\mathcal{X}$, $\mathbf{X}$ is the concatenated matrix in which each row is the row vector of one observation of inputs. Let $\mathbf{f} = [f(\mathbf{x}_1) \ldots f(\mathbf{x}_N)]^\top$. In Gaussian process regression,

$$\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{X}}, K_{\mathbf{X},\mathbf{X}}) \quad \text{implies} \quad \mathbf{y} \mid \mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{X}}, K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N),$$

where $\mathbf{I}_N$ is the identity matrix of size $N$, $\boldsymbol{\mu}_{\mathbf{X}} = [\mu(\mathbf{x}_1), \ldots, \mu(\mathbf{x}_N)]^\top$ is a prior mean vector parameteried by a mean function $\mu\mathcal{X} \mapsto \mathbb{R}$, and $K_{\mathbf{X},\mathbf{X}}$ denotes the covariance matrix characterized by a kernel function $k : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$ such that $K_{\mathbf{X},\mathbf{X}} = [k(\mathbf{x}_i, \mathbf{x}_{i'})]_{i,i'=1\ldots N}$.

Unless there is some extra knowledge about the prior mean function, for simplicity one can choose $\mu(\cdot) = 0$. Note that the convention is related only to the prior distribution and does not imply that the posterior distribution (the prediction) has zero mean. Similarly, we can impose some knowledge of the target function on the choice of the kernel function. For example, the exponential sine squared kernel takes into account the periodic characteristic of the target function ([28], p. 92). The Matern kernel and the squared exponential kernel are the most frequently used stationary kernels ([28], pp. 83–84). Other kernel functions can be found in ([28], Section 4). When $y_i$ in $\mathcal{D}$ is observed directly from the ground truth function $f$, i.e., $y_i = f(\mathbf{x}_i)$, the white noise $\epsilon_i$ in (4) and its variance $\sigma$ in the above equations will be removed, and this refers to interpolation with noise-free observations. In summary, Gaussian processes provide a lot of flexibility in integrating extra knowledge of the learning problem, apart from data set $\mathcal{D}$, into the prior model hypothesis $p(f)$, which is particularly useful for financial applications.

For a matrix of $M$ test points, denoted by $\mathbf{X}^* = [\mathbf{x}_1^*, \ldots, \mathbf{x}_M^*]^T$, the joint prior distribution of the response reads

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}^* \end{bmatrix} \sim \mathcal{N}\left( \begin{matrix} \boldsymbol{\mu}_{\mathbf{X}} \\ \boldsymbol{\mu}_{\mathbf{X}^*} \end{matrix}, \begin{bmatrix} K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N & K_{\mathbf{X},\mathbf{X}^*} \\ K_{\mathbf{X}^*,\mathbf{X}} & K_{\mathbf{X}^*,\mathbf{X}^*} \end{bmatrix} \right),$$

where $\mathbf{f}^* = [f(\mathbf{x}_1^*), \ldots, f(\mathbf{x}_M^*)]^\top$, $\boldsymbol{\mu}_{\mathbf{X}^*} = [\mu(\mathbf{x}_1^*), \ldots, \mu(\mathbf{x}_M^*)]^\top$; and $K_{\mathbf{X}^*,\mathbf{X}^*} = [k(\mathbf{x}_i^*, \mathbf{x}_{i'}^*)]_{i,i'=1\ldots M}$ and $K_{\mathbf{X},\mathbf{X}^*} = K_{\mathbf{X}^*,\mathbf{X}}^\top$ are, respectively, covariance matrix of $\mathbf{X}^*$, and of $\mathbf{X}$ and $\mathbf{X}^*$. Once the model is learned (see Appendix A), the predictive (posterior) distribution of $\mathbf{f}^*$ is

$$\mathbf{f}^* \mid \mathbf{X}, \mathbf{y}, \mathbf{X}^* \sim \mathcal{N}(\mathrm{E}[\mathbf{f}^* \mid \mathbf{X}, \mathbf{y}, \mathbf{X}^*], \mathrm{Var}[\mathbf{f}^* \mid \mathbf{X}, \mathbf{y}, \mathbf{X}^*]),$$

where the posterior mean and variance are given as follows:

$$\begin{aligned} \bar{\mathbf{f}}^* := \mathrm{E}[\mathbf{f}^* \mid \mathbf{X}, \mathbf{y}, \mathbf{X}^*] &= \boldsymbol{\mu}_{\mathbf{X}^*} + K_{\mathbf{X}^*,\mathbf{X}} \left[ K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N \right]^{-1} \mathbf{y}, \\ \mathrm{Var}[\mathbf{f}^* \mid \mathbf{X}, \mathbf{y}, \mathbf{X}^*] &= K_{\mathbf{X}^*,\mathbf{X}^*} - K_{\mathbf{X}^*,\mathbf{X}} \left[ K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N \right]^{-1} K_{\mathbf{X},\mathbf{X}^*}. \end{aligned} \tag{5}$$

**Remark 2.**

***(i)*** *As we discuss in Appendix A, the repeated computation of the inverse matrix* $\left[ K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N \right]^{-1}$*, whose complexity is* $\mathcal{O}(N^3)$ *(using Cholesky decomposition), constitutes a bottleneck in the learning procedure of the GP model. However, in the prediction phase (5), the matrix inversion is required only once and has already computed during the learning phase, making the prediction process significantly faster. In particular, the posterior mean is numerically computed using the following linear expression for* $i' = 1 \ldots M$*:*

$$\bar{\mathbf{f}}_{i'}^* = \mu(\mathbf{x}_{i'}^*) + \sum_{i=1}^{N} \omega_i k(\mathbf{x}_i, \mathbf{x}_{i'}^*),$$

*where* $\boldsymbol{\omega} = [\omega_1, \ldots, \omega_N]^\top = \left[ K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N \right]^{-1} \mathbf{y}$.

***(ii)*** *The posterior mean can be also written by the following linear combination, without the loss of generality. Suppose the zero mean prior is*

$$\bar{\mathbf{f}}_{i'}^* = \sum_{i=1}^{N} \gamma_i y_i, \tag{6}$$

*where* $\boldsymbol{\gamma} = [\gamma_1, \ldots, \gamma_N]^\top = K_{\mathbf{X}^*,\mathbf{X}}\left[K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N\right]^{-1}$. (6) *interprets the GP prediction of the function* $f$ *at a new point, say* $\mathbf{x}^*_{i'}$*, as a linear combination from a set of* $N$ *observable points* $(y_i)_{i=1\ldots N}$*, making it resemble classical interpolation techniques such as splines. However, the GPs model is a complex interpolator due to the nonlinear term incorporated in* $\gamma_i$*. Moreover, when exact observations are not available, as is often the case in real-world applications, the model will interpolate from the noisy observations, taking into account the variance of noise (cf.* (4)*). Hence, the GP model is considered a powerful no-linear interpolation tool.*

### 3.2. Application to Derivative Portfolio Valuation

The application of Gaussian process regression is not limited to evaluating mono-asset derivatives, as illustrated below; it also extends to multi-asset derivatives in the same manner. However, when the number of underlying assets is high, the grid point sampling technique used below will result in exponential growth in complexity, making it necessary to switch to Monte Carlo sampling instead. We also note that the regression model can directly approximate the value of the entire portfolio without any additional computational cost for learning and prediction, regardless of the size and complexity of the portfolio. This observation makes it particularly favorable for risk calculation applications ([30], Remark 3).

When the considered portfolio $V_t$ consists of a possibly large number of mono-asset derivatives, one can still efficiently use GP regression by the decomposition technique provided in [13]. More precisely, while the portfolio may depend on a vector of $d$ asset prices $S = [S^1, \ldots, S^d]$, each derivative is underwritten on a single asset price. The portfolio price $V_t$ is the conditional expectation of the total derivative payoffs given the stock asset vector $S_t$, hence, a function of $S_t$. By the nature of the considered portfolio, we are able to categorize $V_t$ by risk factor

$$V_t = \sum_{j=1}^{d} V_t^j(S_t^j), \tag{7}$$

where $V_t^j$ is the sub-portfolio corresponding to the $j$-th stock. We build the approximate of each $V_t^j$ by one-dimensional Gaussian process model. To this end, for $j = 1, \ldots, d$, we sample $N$ equidistant training points of $S_t^j$, say $\mathbf{s}^j = [s_1^j, \ldots, s_N^j]$, in a determined range $[\underline{s}^j, \bar{s}^j]$, and the corresponding price $\mathbf{v}^j = [V_t^j(s_1^j), \ldots, V_t^j(s_N^j)]$. As this is the one-dimensional interpolation case and the price functions in finance are usually well behaved, we only need to recall the pricing engine few times for sampling training data. For example, in the mono-asset portfolio experiments [13], the GPs can reach good results using only $N = 10$. Once the learning is complete, the GP price of a sub-portfolio at a new point $s^*$ is the predictive mean of the posterior distribution

$$\mathrm{E}[V_t^j(s^*) \mid \mathbf{s}^j, \mathbf{v}^j, s^*] = \sum_{i=1}^{N} \gamma_j V_t^j(s_i^j),$$

where $\gamma_j$ is determined in Remark 2 **(ii)** (with the adaptation of notations). The GP price of the entire portfolio is then deduced by (7). The market risk calculation is made straightforward by using the GP pricing model instead of the pricing engine in the Monte Carlo method (cf. Algorithm 1).

**Remark 3.**

***(i)*** *For a derivative that depends on several market risk factors such as asset volatility, dividend, interest rate, and so on, the bank sometimes uses the Black model for the pricing engine. In this*

*case, we can bring it back to the two-dimensional interpolation problem, i.e., the discounted future price and the volatility, and efficiently apply GP.*

**(ii)** *In the case of basket options, one can use the current price of $\sum_{j=1}^{d} \alpha_j S_t^j$ (cf. Table 1) as the unique learning feature, leading to one-dimension Gaussian process regression.*

**(iii)** *When the derivative depends on many risk factors, i.e., multi-asset products, the algorithm may require a large number of training data for a good approximation. In our financial application, the time to obtain pricing training points (sampled by costly pricing engine) is more relevant than the time to learn the GPR model mentioned in Appendix A.*

## 4. Multi-Fidelity Gaussian Process Regression for Option Pricing

In the previous section, we described how to apply GPR to interpolate the surface of price function using a small number of training price points, thereby enhancing risk calculations in banks. However, the number of training points required to maintain high accuracy grows rapidly with the number of risk factors. This scalability issue becomes practically pronounced when pricing multi-asset options in Section 2.2. While direct application of GPR can still be efficient in moderate dimensions, various enhancement techniques may be employed to sustain accuracy. For instance, in the context of interest rate derivatives, ref. [26] apply dimensionality reduction techniques, e.g., principal component analysis (PCA) or learning from diffusive risk factors [31], to reduce the number of risk factors before interpolating the portfolio value by Chebyshev tensors. However, when products are highly nonlinear and depend on many diffusive risk factors, such as multi-asset derivatives, the effectiveness of such reduction techniques diminishes. This motivates us to explore other techniques such as multi-fidelity modeling.

In financial modeling, it is common to have access to additional sources of information or financial relationships beyond the primary pricing engine. For instance, we may have a simplified pricing engine or "cheap" price data of other highly correlated products or portfolios. The following question then arises: How can we effectively leverage this supplementary information to enhance the modeling? To address this, we explore multi-fidelity Gaussian process regression (mGPR) [12,20,32], a method that allows us to learn the ground truth target from multiple information sources, each with varying levels of fidelity.

### *4.1. Multi-Fidelity Gaussian Process Regression Model*

While multi-fidelity modeling can integrate data and models from multiple fidelity levels, in this study we focus on the case of two fidelity levels. Suppose we have a data set of $N_l$ low-fidelity points, denoted by $\mathcal{D}_l = (\mathbf{X}_l, \mathbf{y}_l) = (\mathbf{x}_i, y_{l_i})_{i=1\ldots N_l}$, and a second set of $N_h (\ll N_l)$ high-fidelity points, denoted by $\mathcal{D}_h = (\mathbf{X}_h, \mathbf{y}_h) = (\mathbf{x}_j, y_{h_j})_{j=1\ldots N_l}$. Ref. [32] used the autoregressive model for the cross-correlation structure between different levels:

$$\begin{cases} f_h(\mathbf{x}) = \rho f_l(\mathbf{x}) + \xi(\mathbf{x}), \quad \mathbf{x} \in \mathcal{X}, \\ y_{l_i} = f_l(\mathbf{x}_i) + \epsilon, \quad \mathbf{x}_i \in \mathbf{X}_1, \\ f_l(\mathbf{x}) \perp \xi(\mathbf{x}) \mid \mathbf{x}, \end{cases} \tag{8}$$

where $\rho$ is where the parameter $\rho$ describes the correlation of the high- and low-fidelity data, $\xi(\mathbf{x})$ measures the mismatch between $\mathbf{f}_h$ and $\rho f_l(\mathbf{x})$, and $\epsilon$ is the white noise of the low-fidelity model. In multi-fidelity modeling, the term $\xi$ is assumed to be conditionally independent of the low-fidelity model $f_l$ given the input $\mathbf{x}$. The goal of the model is to learn the mapping $f_h$ between the input $\mathbf{X}_h$ and the high-fidelity response $\mathbf{y}_h$.

Multi-fidelity modeling can be combined with various regression frameworks to approximate the high-fidelity response, such as neural networks [16] and Gaussian process regression [12,20]. In this work, we investigate the latter by employing multi-fidelity Gaussian process regression (mGPR), which leverages the probabilistic structure of GPR to

jointly model low- and high-fidelity data, thereby enhancing predictive performance while reducing the need for costly high-fidelity evaluations.

In mGPR, $f_l$ is the first Gaussian process model of $\mathbf{y}_l$ against $\mathbf{X}_l$ and $\xi$ is the second GP model conditionally independent with the former. In the following, we consider GP models with constant prior mean [32,33] parameterized by coefficient $\beta$, covariance function $k_{\boldsymbol{\theta}}$, and parameterized by $\boldsymbol{\theta}$, and a multiplicative form of marginal variance $\sigma^2$. In particular, the prior model is

$$\mathbf{f}_l \sim \mathcal{N}(\beta_l, \sigma_l^2 K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)), \tag{9}$$

and

$$\boldsymbol{\xi} \sim \mathcal{N}(\beta_h, \sigma_h^2 K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)), \tag{10}$$

where $\mathbf{f}_l$ and $\boldsymbol{\xi}$ are mean vectors $[f_l(\mathbf{x}_1) \dots f_l(\mathbf{x}_{N_l})]^\top$ and $[\xi(\mathbf{x}_1) \dots \xi(\mathbf{x}_{N_h})]^\top$, and the covariance matrices $K_{\boldsymbol{\theta}_{l(h)}}(\mathbf{X}_{l(h)}, \mathbf{X}_{l(h)}) = [k_{\boldsymbol{\theta}_{l(h)}}(\mathbf{x}_i, \mathbf{x}_{i'})]_{\mathbf{x}_i, \mathbf{x}_{i'} \in \mathbf{X}_{l(h)}}$. Hence, the model is characterized by the parameters of the first and second Gaussian processes: $(\beta_l, \sigma_l, \boldsymbol{\theta}_l)$ and $(\beta_h, \sigma_h, \boldsymbol{\theta}_h)$, and the correlation parameter $\rho$. The following remark discusses the complexity and scalability of mGPR; further details can be found in Appendix B.

**Remark 4.**

***(i)*** *Thanks to the conditional independence assumption (cf. the third line of (8)), the learning phase of mGPR can be decomposed into two separate optimizations. The first one learns the parameters of the low-fidelity model, which involves a matrix inversion with complexity $\mathcal{O}(N_l^3)$. The second one learns the parameters of the high-fidelity model as well as the correlation parameter, involving a matrix inversion with complexity $\mathcal{O}(N_h^3)$. Each of these optimizations can be carried out in the same manner as in standard Gaussian process regression (see the discussion at the end of Appendix A).*

***(ii)*** *When the number of low-fidelity points $N_l$ is large, it becomes relevant to consider simplifications of (10), such as employing a sparse Gaussian process model [34] to reduce computational costs.*

***(iii)*** *Under certain specific setups (see Proposition A1), the matrix inversion involved in the prediction phase has a complexity of $\mathcal{O}(N_l^3 + N_h^3)$.*

### *4.2. Financial Intuition Behind GPR and Multi-Fidelity GPR Estimation*

In market risk management, estimating metrics like value-at-risk (VaR) or expected shortfall (ES) via Monte Carlo simulations involves computing portfolio values under a large number of simulated market scenarios. This requires repeatedly calling pricing functions for all payoffs in the portfolio—each potentially complex and computationally expensive. When a full-fledged Monte Carlo pricer uses, say, 100,000 inner simulations to achieve accurate pricing for exotic multi-underlying instruments, applying it to 10,000 outer market scenarios becomes computationally prohibitive. A Gaussian process regression (GPR) model can serve as a surrogate for the entire portfolio, trained on a small number of scenarios (e.g., 100) evaluated using the full pricer, and then used to predict the remaining valuations at negligible cost. However, even this reduced training set can be expensive if each training sample requires 100,000 inner simulations. To further reduce computational overhead, we propose using multi-fidelity Gaussian process regression (mGPR). The core idea behind mGPR is to combine high-fidelity and low-fidelity data during training:

- A small number of high-quality valuations (e.g., 50 portfolio prices obtained with a pricer using 100,000 simulations);
- A larger number of cheaper, lower-quality valuations (e.g., 50 prices using a pricer with only 5000 simulations).

This combination leverages the correlation structure between low- and high-fidelity models, allowing the surrogate to achieve nearly the same accuracy as a model trained solely on high-fidelity data, but at significantly lower cost. Such a framework is particularly well suited for real-world financial portfolios, where the use of multiple pricing fidelity arises naturally due to time constraints and hardware limitations. Using 100,000 inner paths and evaluating 10,000 outer scenarios would require a billion simulations, justifying the need for a surrogate model trained on a few high-fidelity evaluations and numerous cheap approximations. The mGPR thus offers a powerful compromise between computational efficiency and pricing accuracy, leading to fast and reliable estimation of risk metrics without overloading the pricing infrastructure.

### *4.3. Illustrative Application of Multi-Fidelity Model in American Option Pricing*

We now present another example of the multi-fidelity modeling that explores financial relationships. In the American option pricing case, where the accurate price value is calculated using a binomial tree with 100 time steps, the weaker price can be generated by a sparser tree with only, e.g., 10 time steps. Conversely, if the target is the American option, which is expensively computed by tree model, its European counterpart, which is quickly evaluated using the Black–Scholes formula, can serve as lower-fidelity data.

Figure 2 and Table 2 demonstrate the numerical test of an American put of strike 62. The configuration of this experiment is detailed in Sections 5 and 6.1. In particular, the GPR is trained with only 5 American put prices, while the mGPR integrates additionally 10 European prices as low-fidelity data. All training points are drawn from a regular grid of the underlying asset within an interval $[1, 140]$, and test points are uniformly sampled within the same interval. We observe a significant mismatch of the GPR estimate (cf. blue curve), notably in the in-the-money region, whereas the mGPR estimate (cf. yellow curve) satisfactorily fits the American put price computed using a binomial tree with 100 time steps. Additionally, the mGPR provides smaller prediction uncertainty represented by the yellow shadow interval, which closely aligns with the yellow curve and is visually indistinguishable in Figure 2. For further comparison, we also implement a second standard GPR, learning the discrepancy between the American and European prices, with the European price serving as a control variate to help reduce the variance of the target American price, and the Barone–Adesi and Whaley (BW) approximation [35]. Their results are then reported in Table 2.

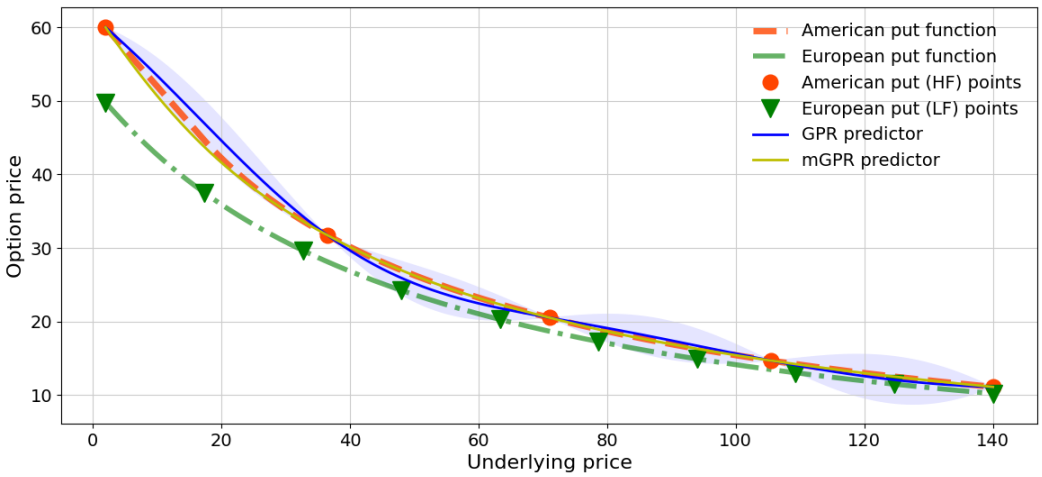


**Figure 2.** American put prices predicted by GPRs learned from 5 points. Red circles represent training points in GPR and high-fidelity points in mGPR, and green inverted triangles are low-fidelity points in mGPR. The blue shadow and yellow shadow (very close to yellow curve) regions depict the uncertainty of the GPR and mGPR predictions, respectively.

**Table 2.** Out-of-sample mean absolute error (MAE) of the predictors against the American put price computed by a binomial tree of 100 time steps. The GPR with control variate learns the discrepancy between the American price and its European counterpart. The BW approximation refers to the Barone-Adesi and Whaley [35] approach.

| | GPR | GPR with Control Variate | BW Approximation | mGPR |
|---|---|---|---|---|
| MAE | 0.6848 | 0.1952 | 0.2859 | 0.1565 |

## 5. Experiment Design and Model Specification

In our experiments, model parameters are randomly generated within realistic market ranges, such as initial prices, volatilities, and covariance eigenvalues, although not calibrated from historical data. To ensure robustness, we test two distinct trading portfolios in different market conditions and periods. We then assess our model's performance against industry benchmarks and advanced machine learning methods using large validation sets. In particular, these experiments are conducted on two trading portfolios (cf. Sections 6.1 and 6.3), interspersed by an examination of individual multi-asset calls (cf. Section 6.2). The first portfolio concerns a portfolio of single underlying derivatives, including plain vanilla (call and put), barrier, and American options. The second portfolio consists of multi-asset options, such as geometric, basket, best-of, and worst-of options. The first single-asset options portfolio, with a time maturity of 10 years, consists of 100 distinct calls and puts issued on each of the four considered underlying assets. In particular, each asset is used to write 10 vanilla, 10 barrier, and 5 American options with their strikes randomly generated from 60 to 140. More details on the market and product parameters in this portfolio can be found in ([13], Tables 1 and A1). We then evaluate the price of each multi-asset option, as indicated in Table 1. Each option expires after two years and is based on the same five assets. The actual values of geometric call options can be calculated using the Black–Scholes formula and serve as a reference.

The second multi-asset options portfolio comprises 500 derivatives, depending on the performance of 20 stocks, generating a total of 20 market risk factors. Each derivative is characterized by various parameters, including a random selection of underlying assets from 2 to 20, a strike price ranging from 80 to 160, and a number of options from 100 to 500. Each derivative can be either a call or a put option among the four types introduced in Section 2.2. The portfolio maturity is set at 5 years, meaning that all its derivatives have a maturity ranging from 6 months to 5 years. In the Black–Scholes model framework, we choose a risk-free rate of 1%. Initial asset prices are arbitrarily chosen within a range of 100 to 120, and the volatility of these assets is set to vary between 0.2 and 0.5 with a random correlation matrix; refer to Table 3 and Figure 3. Finally, we examine the pricing and risk calculation problem for different time horizons, namely, one day, ten days, and one month. At longer horizons, the magnitude of shocks and their volatilities increase, leading to a significant challenge in adapting the number of interpolation points. Financially, the price function becomes less linear and the tail of the potential loss distribution becomes heavier.

**Table 3.** Initial prices of 20 stocks in the multi-asset option portfolio experiment.

| Stock | Initial Price | Stock | Initial Price |
|---|---|---|---|
| $S_0^1$ | 103.79 | $S_0^{11}$ | 115.33 |
| $S_0^2$ | 100.41 | $S_0^{12}$ | 102.28 |
| $S_0^3$ | 109.73 | $S_0^{13}$ | 118.77 |
| $S_0^4$ | 115.22 | $S_0^{14}$ | 118 |
| $S_0^5$ | 108.82 | $S_0^{15}$ | 103.47 |

**Table 3.** *Cont.*

| Stock | Initial Price | Stock | Initial Price |
|---|---|---|---|
| $S_0^6$ | 110.19 | $S_0^{16}$ | 113.28 |
| $S_0^7$ | 100.27 | $S_0^{17}$ | 100.43 |
| $S_0^8$ | 110.23 | $S_0^{18}$ | 102.45 |
| $S_0^9$ | 119.78 | $S_0^{19}$ | 106.64 |
| $S_0^{10}$ | 117.87 | $S_0^{20}$ | 103.32 |

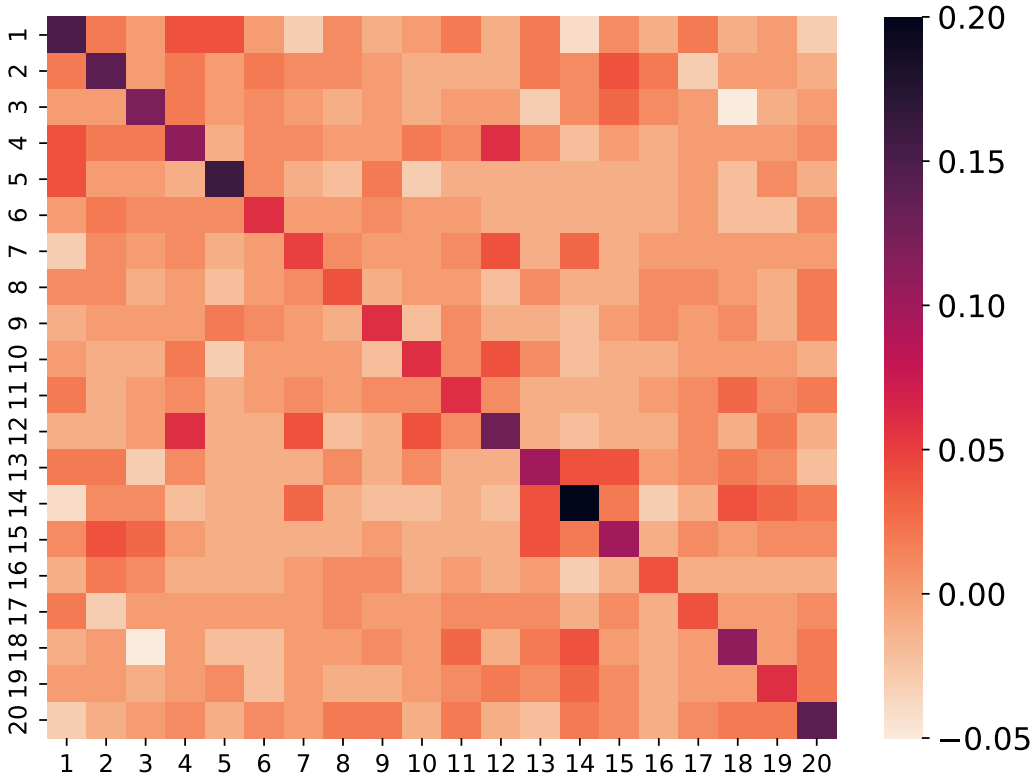


**Figure 3.** Heatmap of the covariance matrix for 20 stocks in the multi-asset option portfolio experiment.

### *5.1. Benchmarking*

For the numerical implementation of the calculation of market risk metrics, the full pricing approach uses $M = 100{,}000$ paths in any case. This level is chosen to ensure a satisfactory level of convergence of high quantiles estimation. However, a limited number ($N \ll M$) of these prices is chosen to train the machine learning models.

Regarding the exact pricing engine, the value of vanilla and barrier options in the mono-asset derivatives portfolio is calculated by analytical formulas, while the binomial tree with 100 time steps is used to evaluate American options. For multi-asset options, the exact pricing engine is defined by a Monte Carlo simulation of 100,000 paths, except for geometric options whose price can be computed by the Black–Scholes formula.

Given this setup, the full repricing approach results in substantial computational demands, a challenge that closely mirrors practical financial risk management. In the case of a multi-asset portfolio with 20 asset prices, the full repricing approach involves the nested calculation of the following elements: 100,000 paths for both the risk metrics and pricing engine, 20 risk factors, 500 products, and 10 time steps. We implement the sensitivity-based pricing approximation (SxS) in Appendix C. To obtain sensitivities data, we apply a relative shock of 0.1% to each risk factor and use the central finite difference, yielding a total of 40 portfolio evaluations for the first-order sensitivity-based approach (see the benchmark bump sensitivity approach in [36]). Including the second-order sensitivities

proves numerically unstable and leads to a worse approximation of the price, primarily due to the nonlinearity of these portfolios. Therefore, they are not reported in our numerical results below.

Regarding machine learning methods, we implement a two-hidden-layer neural network (see Appendix D) (NN) and two Gaussian process models introduced in Section 3 (GPR) and Section 4 (mGPR). In the latter, the high-fidelity data consist of the prices of the Monte Carlo 100,000 paths, while the low-fidelity data comprise weak prices based on the 100 MC paths in Section 6.2 and 500 MC paths in Section 6.3. For measuring the pricing mismatch, we use the mean absolute percentage error (MAPE) over $M = 100{,}000$ scenarios in the VaR and ES computation:

$$\text{MAPE} = \frac{1}{M}\sum_{i=1}^{M}\frac{|\hat{p}_0(S_h) - p_0(S_h)|}{p_0(S_0)},$$

where $p_0(S_h)$ and $\hat{p}_0(S_h)$ are, respectively, actual price (i.e., analytical or Monte Carlo price) and predicted price (e.g., GPR approximative price), and $p_0(S_0)$ is the actual price without shock. The MAPE evaluating the average relative error of the estimate and the true observation is a common choice for benchmarking in finance. For risk assessment purposes, we measure the error of value-at-risk and expected shortfall calculated based on surrogate models (i.e., $\widehat{\text{VaR}}(\cdot, h, \alpha)$ and $\widehat{\text{ES}}(\cdot, h, \alpha)$) against the full revaluation ones calculated based on actual prices (i.e., $\text{VaR}(\cdot, h, \alpha)$ and $\text{ES}(\cdot, h, \alpha)$). For the relative measure, the error is then standardized by the price of today $p_0(S_0)$

$$\text{err. VaR}\,\alpha = \frac{\left|\widehat{\text{VaR}}(\cdot, h, \alpha) - \text{VaR}(\cdot, h, \alpha)\right|}{p_0(S_0)}, \quad \text{err. ES}\,\alpha = \frac{\left|\widehat{\text{ES}}(\cdot, h, \alpha) - \text{ES}(\cdot, h, \alpha)\right|}{p_0(S_0)}. \tag{11}$$

Finally, the computation time is also reported to compare the efficiency of the resource.

### *5.2. Model Specification*

For the neural network model, we train a two-hidden-layer Relu network of 50 hidden units (cf. Appendix D), as we observe numerically that more complex architectures do not improve approximation performance and, instead, make the optimization process significantly more difficult. Network parameters are calibrated by the stochastic gradient descent algorithm, i.e., Adam optimizer [37], using 2000 epochs. The learning rate is set at 0.01, and each batch takes one fifth of the training data size.

We implement standard Gaussian process regression using the scikit-learn Python package (version 1.6.1). A zero-mean prior is assumed, and the covariance structure is modeled using a Matérn kernel with smoothness parameter $\nu = 2.5$. This choice reflects the fact that, as discussed above, the pricing surface is smooth and well behaved, making the Matern class with $\nu = 2.5$ a suitable model. A single length-scale parameter is used, and the input features are standardized by subtracting their empirical mean and dividing by their empirical standard deviation before learning. To ensure numerical stability, the marginal likelihood optimization is restarted five times. Although we tested more complex configurations—including vector-valued length-scales, nontrivial prior mean functions, and additional optimization restarts—these alternatives yielded similar predictive result. We therefore focus on the simple specification, which proved to be both accurate and stable on the provided training data set.

To make the comparison as fair as possible with the standard GPR, the mGPR model configuration is aligned with the standard GPR settings, including the Martern kernel function with $\nu = 2.5$, zero-mean prior and five optimization restarts. Other settings are

left at default as specified by emukit package [38], which is developed by machine learning researchers at the University of Sheffield.

These modeling choices —namely, the use of a large number of training epochs for the neural network, and multiple optimization restarts for the Gaussian process models—are designed to mitigate the randomness inherent in stochastic optimization methods. As a result, the comparison of performance across models remains robust and reliable.

## 6. Numerical Results

The main aim of this section is to provide a numerical analysis of algorithms based on Gaussian process regression (GPR) and the multi-fidelity model (mGPR) when applied to the repeated valuation and market risk calculation of an equity trading portfolio. It is worth emphasizing that a key advantage of the proposed new methods is that valuation and risk assessment are performed at the portfolio level, rather than on a deal-by-deal basis, which is the case with traditional approaches.

### *6.1. Mono-Asset Options Portfolio Case*

Figure 4 and Table 4 outline the performance of Gaussian process regression (GPR) in the full revaluation and risk calculation of a mono-asset portfolio with the sub-portfolio techniques outlined in Section 3.2. The accuracy of the GPR is evident in both pricing and risk calculation. For this mono-asset experiment, we train the regression models by a regular grid and backtest with more (uniformly) random data [10,13]. In particular, the training and test stock prices are shocked around their initial values using their diffusion models, e.g.,

$$\left[S_0^i \exp\left((r - \frac{(\sigma^i)^2}{2})h - 3\sqrt{h}\sigma^i\right), S_0^i \exp\left((r - \frac{(\sigma^i)^2}{2})h + 3\sqrt{h}\sigma^i\right)\right],$$

where $S_0^i, \sigma^i, r$ and $h$ denote, respectively, the initial stock price and volatility of the $i$-th asset, and the common interest rate and time step.

In Figure 4, a precise fit of the price function is achieved when employing five points or more for training each sub-portfolio. The risk measures estimated by the GPR approach consistently converge to the benchmark measures obtained by the full revaluation approach, i.e., the intensive Monte Carlo method with 100,000 paths. The latter takes approximately 2 h and 25 min on our machine. When learning from 10 points (respectively, 5 points), the GPR achieves a speed-up of 1000 times (respectively, 2000 points) compared to the full revaluation approach. The high runtime of the full revaluation approach in this experiment is primarily due to the American pricing calculations using a binomial tree with 100 time steps. This section concludes the performance of the Gaussian process regression in the mono-asset portfolio pricing and risk calculation.

**Table 4.** VaR and ES estimates using GPR against the true measures in the mono-asset portfolio case.

| | **Confidence Level** | **True Measure** | $N = 5$ | $N = 10$ | $N = 20$ |
|---|---|---|---|---|---|
| VaR | 90% | 39.83 | 39.84 | 39.83 | 39.83 |
| | 95% | 50.90 | 50.89 | 50.92 | 50.91 |
| | 97.5% | 60.28 | 60.29 | 60.29 | 60.29 |
| | 99% | 70.53 | 70.55 | 70.56 | 70.54 |
| ES | 90% | 53.98 | 53.98 | 53.99 | 53.98 |
| | 95% | 63.02 | 63.02 | 63.03 | 63.02 |
| | 97.5% | 70.95 | 70.95 | 70.97 | 70.96 |
| | 99% | 80.15 | 80.15 | 80.17 | 80.15 |
| | Speed-up | 2 h 25—benchmark | ×2000 | ×1000 | ×500 |

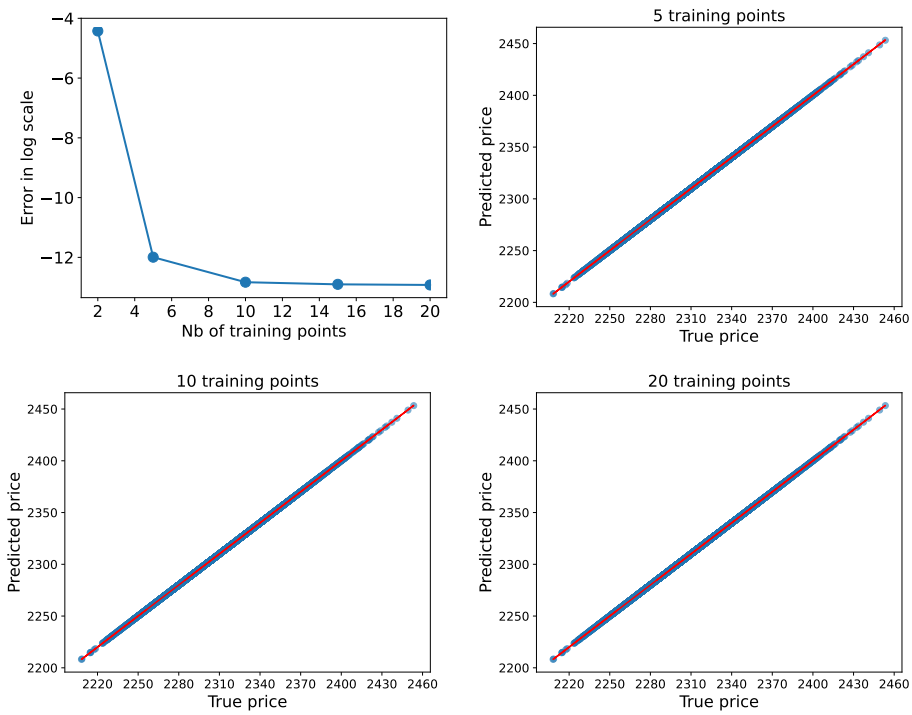


**Figure 4.** Convergence of MAPE (in log scale) of GPR estimates when increasing the number of training points in the mono-asset portfolio case (**top left**). Q–q plots of GPR predicted price using 5 (**top right**), 10 (**bottom left**), and 20 (**bottom right**) training points against the true price, i.e., analytical and binomial tree prices, in the mono-asset portfolio case.

### *6.2. Multi-Asset Options Case*

Figure 5 and Table 5 display the results of the pricing and risk calculation for a five-asset geometric average call. Both GPR and mGPR models are trained using Monte Carlo prices with 100,000 paths, while the analytical prices are used as a reference for backtesting. Additionally, mGPR uses 200 prices obtained through the Monte Carlo method with 100 paths as low-fidelity data, which is still less expensive than one price point used 100,000 paths. The top left plot in Figure 5 shows that two GPR models trained with 20 points can outperform Monte Carlo prices. Notably, mGPR slightly outperforms GPR at lower numbers of data points before their error curves converge.

The quantile–quantile (q–q) plots in Figure 6 trace the left tail of the simulated price distribution below the 10th percentile, corresponding to the right tail of the loss in the risk calculation in Table 5. Despite the Monte Carlo price having a higher MAPE error compared to GPR prices, its tail perfectly matches the true price. The tails of GPR and mGPR prices also converge as the number of training points increases. We stop the convergence curve of mGPR at 200 training points (corresponding to high-fidelity data) because we only use 200 low-fidelity points. In Table 5, the one-day VaR 99% and ES 97.5% estimated by GPRs using 200 data points have an error of around 10 basis points. Notably, GPR models significantly save time in risk calculation, including sampling and training time with 200 points (13 s for GPR compared to 7488 s for the brute force Monte Carlo approach). The mGPR model remains an effective approximation for risk measurement, especially in cases of high pricing engine costs, such as with complex portfolios, and when using minimal price points is preferred.

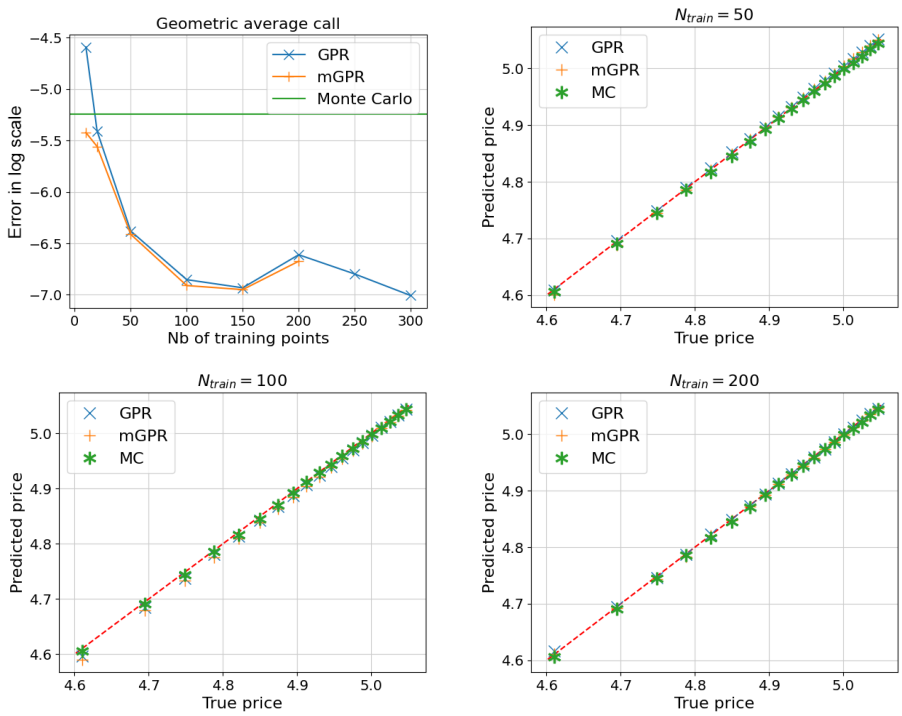


**Figure 5.** MAPE of estimates of **five-asset geometric average call** as a function of the number of training points (**top left**) and q–q plots comparing predictors learned from 50 (**top right**), 100 (**bottom left**), and 200 (**bottom right**) points against the true price, i.e., Black–Scholes price, below their 10% quantile levels. The mGPR uses 200 low-fidelity points.

**Table 5.** One-day VaR 99% and ES 97.5% of **five-asset geometric average call** estimated by predicted pricing models. Computational times (with speed-ups inside parentheses) measure the total times of sampling and learning.

| | **Model** | **Number of Training Points (*N*)** | | | | | |
|---|---|---|---|---|---|---|---|
| | | **10** | **20** | **50** | **100** | **150** | 200 |
| VaR 99% | GPR | 0.7531 | 0.7936 | 0.8035 | 0.8144 | 0.8131 | 0.8039 |
| | mGPR | 0.8173 | 0.8068 | 0.8109 | 0.8202 | 0.8161 | 0.8072 |
| | MC | | | 0.8076 | | | |
| | True | | | 0.8045 | | | |
| ES 97.5% | GPR | 0.7528 | 0.7943 | 0.8056 | 0.8165 | 0.8155 | 0.8048 |
| | mGPR | 0.8191 | 0.8100 | 0.8113 | 0.8227 | 0.8186 | 0.8084 |
| | MC | | | 0.8090 | | | |
| | True | | | 0.8058 | | | |
| Computational time (in second) | GPR | 1 (×7488) | 1 (×7488) | 3 (×2496) | 6 (1248) | 10 (×749) | 13 (×576) |
| | mGPR | 10 | 10 | 12 | 23 | 30 | 32 |
| | MC | | | 7488 | | | |
| True initial price | | | | 5.5355 | | | |

Figures 6–8 and Tables 6–8 show the results for other options, with similar computational times to those in Table 5. Most of the remarks made in the geometric average call case are still valid for these other cases. In the arithmetic average basket call case, the risk measures computed by two GPR models have an error of about 23 basis points, which is less than 10 basis points in the two other cases. These above errors can be computed by applying results in Tables 6–8 into (11).

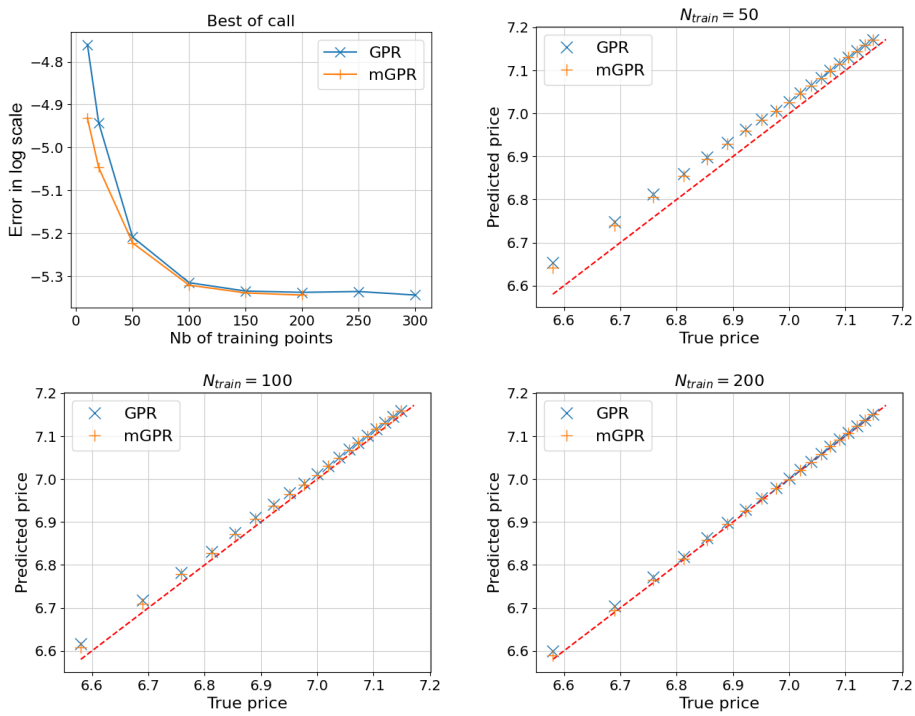


**Figure 6.** MAPE of estimates of **five-asset basket call** as a function of the number of training points (**top left**) and q–q plots comparing predictors learned from 50 (**top right**), 100 (**bottom left**), and 200 (**bottom right**) points against the true price, i.e., Monte-Carlo price with 100,000 paths, below their 10% quantile levels.

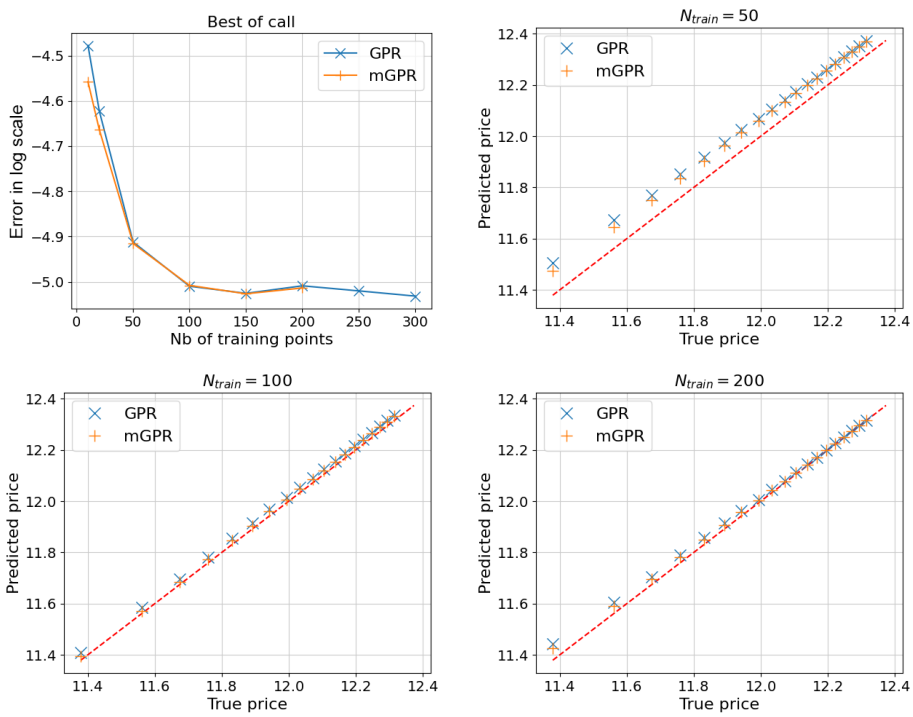


**Figure 7.** MAPE of estimates of **five-asset best-of-call** as a function of the number of training points (**top left**) and q–q plots comparing predictors learned from 50 (**top right**), 100 (**bottom left**), and 200 (**bottom right**) points against the true price, i.e., Monte-Carlo price with 100,000 paths, below their 10% quantile levels.

**Table 6.** One-day VaR 99% and ES 97.5% of **five-asset basket call** estimated by proxy pricing models.

| | **Model** | **Number of Training Points (*N*)** | | | | | |
|---|---|---|---|---|---|---|---|
| | | **10** | **20** | **50** | **100** | **150** | 200 |
| VaR 99% | GPR | 0.9163 | 1.0067 | 1.0289 | 1.0597 | 1.0588 | 1.0732 |
| | mGPR | 1.0123 | 1.0550 | 1.0371 | 1.0673 | 1.0630 | 1.0824 |
| | True | | | 1.1013 | | | |
| ES 97.5% | GPR | 0.9199 | 1.0077 | 1.0280 | 1.0615 | 1.0607 | 1.0748 |
| | mGPR | 1.0143 | 1.0583 | 1.0371 | 1.0674 | 1.0643 | 1.0832 |
| | True | | | 1.1016 | | | |
| True initial price | | | | 7.7770 | | | |

**Table 7.** One-day VaR 99% and ES 97.5% of **five-asset best-of-call** estimated by proxy pricing models.

| | **Model** | **Number of Training Points (*N*)** | | | | | |
|---|---|---|---|---|---|---|---|
| | | **10** | **20** | **50** | **100** | **150** | 200 |
| VaR 99% | GPR | 1.4704 | 1.5850 | 1.6380 | 1.7258 | 1.6907 | 1.7064 |
| | mGPR | 1.5869 | 1.6604 | 1.6682 | 1.7403 | 1.6982 | 1.7346 |
| | True | | | 1.7483 | | | |
| ES 97.5% | GPR | 1.4714 | 1.5878 | 1.6435 | 1.7279 | 1.6928 | 1.7084 |
| | mGPR | 1.5906 | 1.6675 | 1.6721 | 1.7438 | 1.7010 | 1.7373 |
| | True | | | 1.7517 | | | |
| True initial price | | | | 13.3101 | | | |

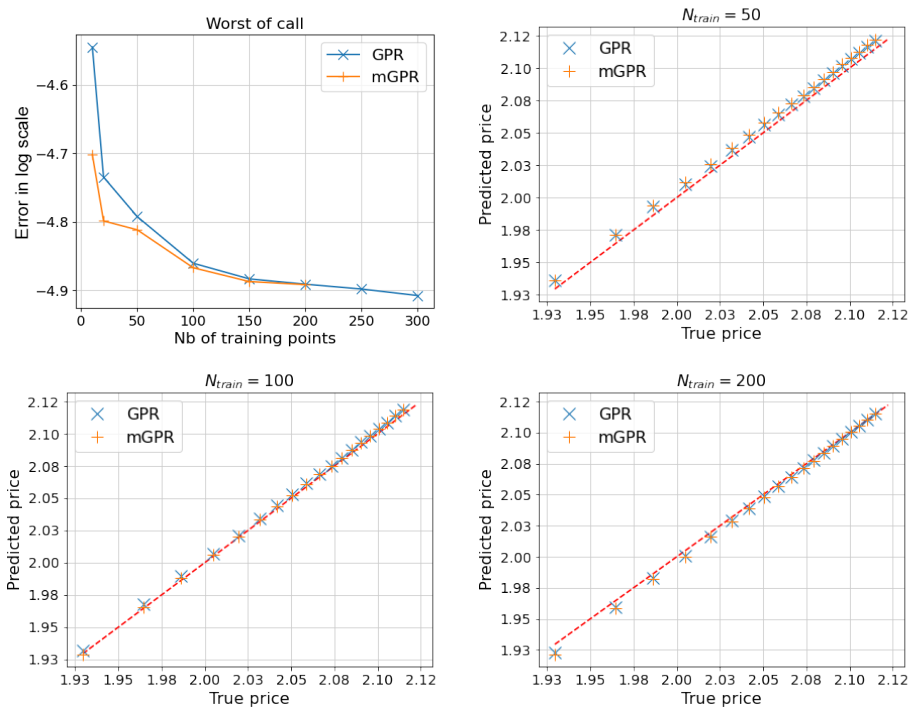


**Figure 8.** MAPEof estimates of **five-asset worst-of call** as a function of the number of training points (**top left**) and q–q plots comparing predictors learned from 50 (**top right**), 100 (**bottom left**), and 200 (**bottom right**) points against the true price, i.e., Monte-Carlo price with 100,000 paths, below their 10% quantile levels.

**Table 8.** One-day VaR 99% and ES 97.5% of **five-asset worst-of call** estimated by proxy pricing models.

| | **Model** | **Number of Training Points (*N*)** | | | | | |
|---|---|---|---|---|---|---|---|
| | | **10** | **20** | **50** | **100** | **150** | 200 |
| VaR 99% | GPR | 0.3153 | 0.3447 | 0.3584 | 0.3619 | 0.3661 | 0.3704 |
| | mGPR | 0.3625 | 0.3673 | 0.3584 | 0.3640 | 0.3677 | 0.3711 |
| | True | | | 0.3651 | | | |
| ES 97.5% | GPR | 0.3625 | 0.3673 | 0.3584 | 0.3640 | 0.3677 | 0.3711 |
| | mGPR | 0.3624 | 0.3670 | 0.3587 | 0.3645 | 0.3681 | 0.3715 |
| | True | | | 0.3657 | | | |
| True initial price | | | | 2.3296 | | | |

In summary, for any type of product, both GPR and mGPR offer significant speed-ups in risk calculation, compared to the traditional full revaluation approach, while still maintaining a high level of accuracy in their approximations. Although mGPR requires slightly more computational time than GPR, this additional time becomes negligible when compared to the time required for the full revaluation approach. Moreover, mGPR delivers a more accurate approximation, particularly when the number of training points is small. As the number of training points increases, however, the performance difference between the two GPR models diminishes, and they converge in terms of approximation quality.

### *6.3. Multi-Asset Options Portfolio Case*

Figure 9 and Table 9 illustrate the results of a multi-asset options portfolio. The time horizon of the value-at-risk and expected shortfall is set at one day. The second-order sensitivity-based approach (SxS) and the neural network learning from 40 points are not reported here due to their low precision. The learning of mGPR incorporates 500 prices computed by 500 Monte Carlo paths as low-fidelity data. To verify the robustness of the three regression methods, we plot the convergence curve in the top left plot of Figure 9, redrawing the training data sets 10 times. The curve represents the mean of the MAPE, while the error bars indicate the standard error of the MAPE. For the sake of representation, other plots in Figure 9 and the results in Table 9 correspond to one training instance, providing a snapshot of the performance of the methods under a single set of training conditions.

In the top left panel of Figure 9, the convergence curves for GPR and mGPR are significantly better than the NN convergence curve at every point. Despite the portfolio complexity, both GPR models (GPR and mGPR) achieve impressive results with only 40 points. In the first line of Table 9, the GPR model has an MAPE of 0.61%, slightly outperformed by the mGPR model with an MAPE of 0.57%, while SxS performs about 10 times worse with an error of 6.13%. Figure 9 highlights this difference, showing that the predicted left tail of SxS prices diverges significantly from the true price represented by the red diagonal line. While the predicted left tails of GPR and mGPR prices are not perfect, they still concentrate around the tail of the true price, i.e., the red line.

In Table 9, mGPR predictions offer more accurate estimates of the VaR 99% and ES 97.5% compared to GPR predictions, with errors of 34 bps and 32 bps, respectively, versus 54 bps for both errors in GPR. As more data become available, the performance of all models improves. Generally, two GPR models outperform NN, with an MAPE of 0.42% (0.38%) compared to that of NN's 0.5% (0.43%) when learning with $N = 100$ ($N = 500$). The difference is more pronounced in the prediction of the left tail, as seen in the two bottom plots in Figure 9. The left tails predicted by GPR and mGPR closely align with

the true one, while NN underestimates the left tail. When trained with $N = 100$, mGPR's errors in risk measures, around 30 bps, are only half of GPR's errors. However, this difference disappears when training with $N = 500$. Tables 10 and 11 and Figures 10 and 11 correspond to the risk assessment results for a ten-day and one-month horizon, and observations from the one-day horizon case hold for other time horizons.

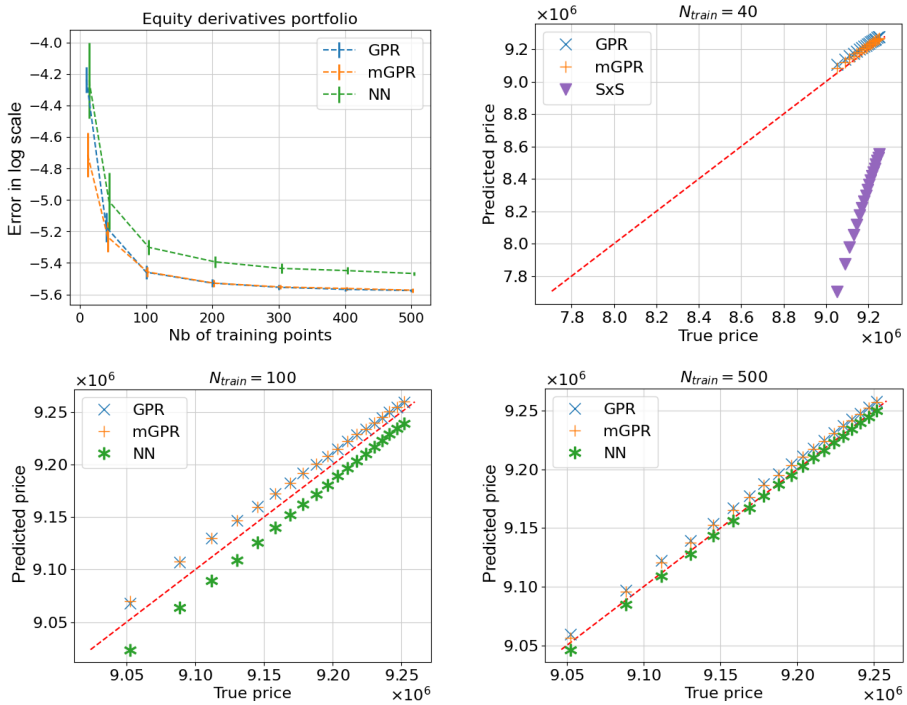


**Figure 9.** MAPE of estimates for an equity option portfolio with a **one-day horizon** as a function of the number of training points (**top left**). The error bars represent the standard error, calculated by rerunning the learning process on 10 redrawn training sets. Q–q plots comparing predictors learned from 40 (**top right**), 100 (**bottom left**), and 500 (**bottom right**) points against the true price, i.e., Monte-Carlo price with 100,000 paths, below their 10% quantile levels. The MAPE of SxS prediction is not included in figures due to its high value, but it can be found in Table 9.

**Table 9.** Pricing approximation and risk calculation of the portfolio with **one-day horizon**.

| ***N*** | **Full Pricing** | **40** | | | | **100** | | | **500** | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Model** | | **SxS** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** |
| MAPE | 9,447,616 | 6.13% | 0.61% | **0.57%** | 0.48% | 0.43% | **0.42%** | 0.43% | **0.38%** | **0.38%** |
| VaR 99% | 358,862 | 1,575,039 | 307,983 | 325,145 | 374,986 | 340,394 | 343,926 | 368,887 | 350,397 | 351,493 |
| ES 97.5% | 359,972 | 1,584,907 | 309,301 | 328,193 | 377,288 | 342,130 | 347,312 | 370,475 | 351,211 | 353,116 |
| Err. VaR 99% | - | 12.87% | 0.54% | **0.36%** | 0.17% | 0.20% | **0.16%** | 0.11% | 0.09% | **0.08%** |
| Err. ES 97.5% | - | 12.97% | 0.54% | **0.34%** | 0.18% | 0.19% | **0.13%** | 0.11% | 0.09% | **0.07%** |

The lowest errors for each training dataset are are emphasized in bold.

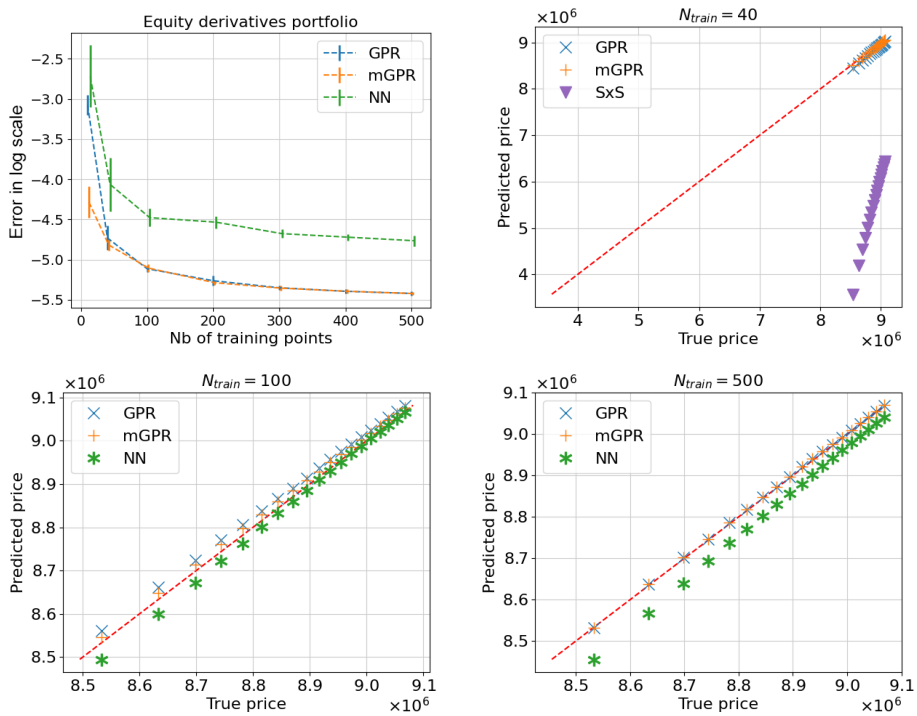


**Figure 10.** MAPE of estimates for an equity option portfolio with a **ten-day horizon** as a function of the number of training points (**top left**). The error bars represent the standard error, calculated by rerunning the learning process on 10 redrawn training sets. Q–q plots comparing predictors learned from 40 (**top right**), 100 (**bottom left**), and 500 (**bottom right**) points against the true price, below their 10% quantile levels. The MAPE of SxS prediction is not included in figures due to its high value, but it can be found in Table 10.

**Table 10.** Pricing approximation and risk calculation of the portfolio with **ten-day horizon**.

| *N* | Full Pricing | 40 | | | | 100 | | | 500 | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Model** | | **SxS** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** |
| MAPE | 9,447,616 | 20.27% | 1.18% | **0.92%** | 1.19% | 0.64% | **0.61%** | 0.8% | **0.44%** | **0.44%** |
| VaR 99% | 814,166 | 5,252,412 | 896,191 | 843,186 | 835,971 | 785,850 | 799,501 | 847,257 | 810,857 | 811,540 |
| ES 97.5% | 814,604 | 5,320,871 | 895,574 | 843,831 | 836,584 | 787,204 | 800,163 | 848,514 | 812,423 | 812,947 |
| Err. VaR 99% | - | 46.98% | 0.87% | **0.31%** | 0.23% | 0.30% | **0.16%** | 0.35% | **0.04%** | **0.03%** |
| Err. ES 97.5% | - | 47.70% | 0.86% | **0.31%** | 0.23% | 0.29% | **0.15%** | 0.36% | **0.02%** | **0.02%** |

The lowest errors for each training dataset are are emphasized in bold.

**Table 11.** Pricing approximation and risk calculation of the portfolio with **one-month horizon**.

| *N* | Full Pricing | 40 | | | | 100 | | | 500 | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Model** | | **SxS** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** |
| MAPE | 9,447,616 | 38.44% | 2.35% | **1.50%** | 9.48% | **1.02%** | 1.09% | 1.72% | **0.60%** | **0.60%** |
| VaR 99% | 1,016,862 | 10,137,189 | 1,175,837 | 1,046,029 | 1,092,622 | 995,794 | 1,006,735 | 1,103,964 | 1,009,737 | 1,010,166 |
| ES 97.5% | 1,011,890 | 10,267,028 | 1,177,443 | 1,042,407 | 1,104,343 | 991,431 | 1,004,123 | 1,119,376 | 1,005,141 | 1,005,065 |
| Err. VaR 99% | | 96.54% | 1.68% | **0.31%** | 0.8% | 0.22% | **0.11%** | 0.92% | **0.08%** | **0.07%** |
| Err. ES 97.5% | | 97.96% | 1.75% | **0.32%** | 0.98% | 0.22% | **0.08%** | 1.14% | **0.07%** | **0.07%** |

The lowest errors for each training dataset are are emphasized in bold.

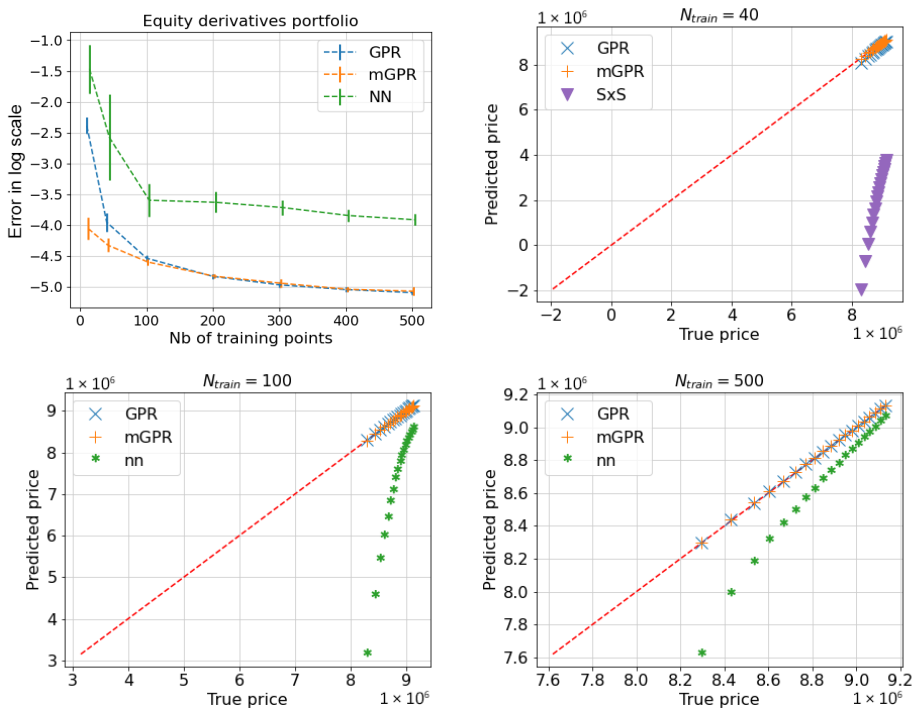


**Figure 11.** MAPE of estimates for an equity option portfolio with a **one-month horizon** as a function of the number of training points (**top left**). The error bars represent the standard error, calculated by rerunning the learning process on 10 redrawn training sets. Q–q plots comparing predictors learned from 40 (**top right**), 100 (**bottom left**), and 500 (**bottom right**) points against the true price, i.e., Monte-Carlo price with 100,000 paths, below their 10% quantile levels. The MAPE of SxS prediction is not included in figures due to its high value, but it can be found in Table 11.

Table 12 shows the runtime of different risk calculation methods on a server with an Intel(R) Xeon(R) Gold 5217 CPU and an Nvidia Tesla V100 GPU. The full repricing method, involving 100,000 shock scenarios and Monte Carlo simulations with 100,000 paths over 500 derivatives, can take up to a full day without GPU acceleration—often unavailable in banking environments—making it impractical for daily use.

In contrast, alternative methods offer substantial speed-ups. Standard Gaussian process regression (GPR) is 290 times faster than full repricing with $N = 100$ training points (13 vs. 3777 s). While multi-fidelity GPR (mGPR) has longer training time (56 s), it remains highly efficient and more accurate, and especially valuable when pricing engine usage is limited and computational resources are constrained. It is worth noting that this advantage becomes even more pronounced under the realistic constraints of possibly slower pricing engines typically used in banks.

**Table 12.** Computation time comparison of VaR and ES calculation by alternative methods.

| $N$ | Full Pricing | 40 | | | | 100 | | | 500 | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Model** | | **SxS** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** | **NN** | **GPR** | **mGPR** |
| Learning time | 0 | 0 | 1 | 41 | 25 | 2 | 56 | 30 | 13 | 135 |
| Sampling time | 3777 | | 5 | | | 11 | | | 43 | |

Much like the single multi-asset option case in Section 6.2, both GPR and mGPR models consistently offer significant speed-ups in risk calculation across multiple risk horizons. With the same number of pricing engine recalls (cf. $N = 40$ training points), GPR models provide significantly higher precision than the traditional SxS approach. Additionally,

when using a modest number of training points (limited to 500), GPR models not only deliver more accurate approximations but also exhibit smaller variance with respect to changes in the training data set.

## 7. Conclusions

We introduced statistical and machine learning tools to handle the repeated valuation of extensive portfolios comprising linear and nonlinear derivatives. To be more precise, we applied a Bayesian nonparametric technique known as Gaussian process regression (GPR). To evaluate the accuracy and performance of these algorithms, we considered a multi-asset options portfolio and a portfolio of nonlinear derivatives, including vanilla and barrier/American options on an equity asset. The numerical tests demonstrated that the GPR algorithm outperforms pricing models in efficiently conducting repeated valuations of large portfolios. It is noteworthy that the GPR algorithm eliminates the need to separately revalue each derivative security within the portfolio, leading to a significant speed-up in calculating value-at-risk (VaR) and expected shortfall (ES). This independence from the size and composition of the trading portfolio is remarkable. Consequently, we argue that it is more advantageous for banks to construct their risk models using the power of GPR techniques in terms of calculation accuracy and speed-up. Moreover, we investigated the multi-fidelity modeling technique and explored its applications in pricing and risk calculation, a relatively novel concept in quantitative finance. The multi-fidelity modeling approach leverages limited access to the price engine while incorporating information from other related, more affordable resources to enhance the approximation. Trained regression models offer price estimates at the portfolio level, making them more favorable for risk calculation compared to classical pricing approaches that evaluate products individually. Our numerical findings indicate that machine learning models such as neural networks and Gaussian process regression provide more accurate results and significant time savings compared to traditional sensitivity-based approaches, maintaining precision in risk calculation akin to the full repricing approach. With the limitation of training price data, GPR models yield more impressive results than neural networks. Despite the slightly longer learning time required for mGPR, especially when low-fidelity data are available, it yields better, or at least equal, precision compared to the standard GPR model. Notably, as multi-fidelity modeling is not yet well known in quantitative finance, it presents an intriguing direction for research, allowing scientists to explore traditional relationships in finance to define low-fidelity models and enhance the learning process.

Future research delves into other topics and applications of algorithms such as Gaussian process regression and multi-fidelity modeling, with a particular focus on the fixed income portfolio. We also envisage the use of the replicating portfolio value, assumed to be accessible, as low-fidelity data to leverage the learning of the target portfolio value. Additionally, the price of European basket options can serve as low-fidelity data to approximate the price of their American/Bermudian counterparts. The Black–Scholes price of derivatives can be used to improve the approximation of their Heston price. We leave these interesting applications there for future study.

**Author Contributions:** The conceptualisation was carried out by H.D.N. and N.L.; The methodology was developed by N.L. and the formal analysis was conducted by P.O. All authors have read and agreed to the published version of the manuscript.

**Funding:** The research of H.D. Nguyen is funded by a CIFRE grant from Natixis.

**Data Availability Statement:** The data in this study is generated using openly available code from https://github.com/hoangdungnguyen/GPR-mGPR-risk-calculation, accessed on 12 May 2025.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Appendix A. Estimation of Parameters in Gaussian Process Regression

The learning procedure of the Gaussian process model involves finding the best suited parameters of the mean function $\boldsymbol{\mu}_{\mathbf{X}}$, the covariance kernel function $k$, jointly denoted by $\boldsymbol{\theta}$ and the white noise variance $\sigma$. The set of GP parameters can be estimated either by maximum likelihood or maximum a posteriori, or the cross-validation method (see ([28], Section 5.4) and ([29], Section 15.2.4)). By means of computational facility, maximum likelihood is the most used among these and it is chosen to be a default method for the standard Gaussian processes. The criterion $\mathcal{L}$ that we aim to maximize is the marginal likelihood, also called by the (model) evidence reads

$$\mathcal{L}(\mu, \sigma, \boldsymbol{\theta}) = \log \mathrm{P}(\mathbf{y}|\mathbf{X}) = -\Big[(\mathbf{y} - \boldsymbol{\mu}_{\mathbf{X}})^\top \Big(K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N\Big)^{-1} (\mathbf{y} - \boldsymbol{\mu}_{\mathbf{X}}) + \log\det\Big(K_{\mathbf{X},\mathbf{X}} + \sigma^2 \mathbf{I}_N\Big)\Big] + \mathit{constant}.$$

Notice that the first term of (A1) corresponding to the data-fit can be seen as the negative squared loss with a kernel-based weight in linear regression, whereas the second term relates to the penalty of the model complexity (see ([28], Sections 2.4 and 5.4.1)).

Maximizing the evidence (or minimizing the negative evidence) is usually solved by a local optimization method, namely, BFGS ([39], Chapter 6), as in Scikit-learn Python package or by any gradient-based method such as Adam [37] in Gpytorch Python package. In any case, the numerical implementation must be performed by full batch and involves reversing the covariance matrix of size $N \times N$ whose complexity is $\mathcal{O}(N^3)$ (using Cholesky decomposition). Contrarily, the prediction step (cf. (5)) costs only $\mathcal{O}(N^2)$. Hence, even though the Gaussian process model is a great interpolation tool, the classical model is not appropriate for high-dimensional problems that require an important number of training points $N$ to explore the target surface. Many innovative approaches are proposed to tackle this issue, and, in general, they are developed through the use of low-rank approximation (or possibly decomposition) of the covariance matrix [34,40] and/or the use of other learning methods such as (stochastic) variational inference [34,41,42]. We refer the reader to the documentation of Gpytorch package on Python [11] for the details as well as numerical implementation of these approaches.

## Appendix B. Estimation of Parameters in Multi-Fidelity Gaussian Process Regression

### *Appendix B.1. Conditional Distribution of the Estimate*

We now describe the posterior mean and variance of the prediction once the model (8) and (10) is completely learned. Following the demonstration in ([33], Appendix A), for a new point $\mathbf{x}^* \in \mathcal{X}$, the prediction of the high-fidelity model follows normal distribution:

$$f_h(\mathbf{x}^*) \mid \mathbf{f}_l = \mathbf{y}_l, \mathbf{f}_h = \mathbf{y}_h, (\beta_l, \sigma_l, \boldsymbol{\theta}_l), (\rho, \beta_h, \sigma_h, \boldsymbol{\theta}_h) \sim \mathcal{N}(m_h(\mathbf{x}^*), s_h^2(\mathbf{x}^*)),$$

with mean and variance functions:

$$\begin{aligned} m_h(\mathbf{x}^*) &= \beta + K(\mathbf{x}^*)^\top \mathbf{K}^{-1}(\mathbf{y} - \beta), \\ s_h^2(\mathbf{x}^*) &= \rho^2 \sigma_l^2 + \sigma_h^2 - K(\mathbf{x}^*)^\top \mathbf{K}^{-1} K(\mathbf{x}^*), \end{aligned} \tag{A1}$$

where

$$\beta = \frac{\mathbf{1}^\top \mathbf{K}^{-1} \mathbf{y}}{\mathbf{1}^\top \mathbf{K}^{-1} \mathbf{1}}, \qquad \text{with} \qquad \mathbf{y} = \begin{bmatrix} \mathbf{y}_l \\ \vdots \\ \mathbf{y}_h \end{bmatrix}.$$

In addition, $K(\mathbf{x}^*)^\top$ is the covariance function between $f_h(\mathbf{x}^*)$ and $[f_h(\mathbf{x}_1)\dots f_h(\mathbf{x}_{N_l}), f_h(\mathbf{X}_h)]$

$$K(\mathbf{x}^*)^\top = \Big[\rho\sigma_l^2 k_{\boldsymbol{\theta}_l}(\mathbf{x}^*,\mathbf{X}_l) \quad \rho^2\sigma_l^2 k_{\boldsymbol{\theta}_l}(\mathbf{x}^*,\mathbf{X}_h) + \sigma_h^2 k_{\boldsymbol{\theta}_h}(\mathbf{x}^*,\mathbf{X}_h)\Big],$$

and the covariance matrix $\mathbf{K}$ of vector $(\mathbf{f}_l, \mathbf{f}_h)$ reads

$$\mathbf{K} = \begin{pmatrix} \sigma_l^2 K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l) & \rho\sigma_l^2 K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_h) \\ \rho\sigma_l^2 K_{\boldsymbol{\theta}_l}(\mathbf{X}_h,\mathbf{X}_l) & \rho^2\sigma_l^2 K_{\boldsymbol{\theta}_l}(X_h,X_h) + \sigma_h^2 K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h) \end{pmatrix}. \tag{A2}$$

We can see that the computation of (A1) involves reversing the massive covariance matrix $\mathbf{K}$ of size $(N_l + N_h) \times (N_l + N_h)$. To reduce the computational complexity, one can apply the Schur complement of block matrix $\mathbf{K}$ which is presented by Proposition A1.

**Proposition A1** (Proposition 3.1 in [12])**.** *If the high-fidelity input $\mathbf{X}_h$ is a subset of the low-fidelity input $\mathbf{X}_l$, we can sort the data such that $\mathbf{X}_l = (\mathbf{X}_l \backslash \mathbf{X}_h, \mathbf{X}_h)$ or*

$$\mathbf{X}_l = \begin{pmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{N_l-N_h+1}^\top \\ \vdots \\ \mathbf{x}_{N_l}^\top \end{pmatrix} \qquad \textit{and} \qquad \mathbf{X}_h = \begin{pmatrix} \mathbf{x}_{N_l-N_h+1}^\top \\ \vdots \\ \mathbf{x}_{N_l}^\top \end{pmatrix}. \tag{A3}$$

*In the above setup, the inverse matrix of $\mathbf{K}$ in (A2) has the following form:*

$$\mathbf{K}^{-1} = \begin{pmatrix} \sigma_l^{-2} K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l)^{-1} + \begin{pmatrix} 0 & 0 \\ 0 & \rho^2\sigma_h^{-2} K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1} \end{pmatrix} & -\begin{pmatrix} 0 \\ \rho\sigma_h^{-2} K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1} \end{pmatrix} \\ -\begin{pmatrix} 0 & \rho\sigma_h^{-2} K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1} \end{pmatrix} & \sigma_h^{-2} K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1} \end{pmatrix}. \tag{A4}$$

Instead of performing the inversion matrix in (A2), which has a complexity of $\mathcal{O}((N_l + N_h)^3)$, the computation of $\mathbf{K}^{-1}$ in (A4), involving the inverse of $K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l)^{-1}$ and $K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1}$, has a much lower complexity of $\mathcal{O}(N_l^3 + N_h^3)$. By substituting (A4) into (A1) and performing some calculus, one can simplify the above formulas as follows:

$$\beta = \frac{\sigma_l^{-2}\mathbf{1}^\top K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l)^{-1}\mathbf{y}_l + (1-\rho)\sigma_h^{-2} K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1}(\mathbf{y}_h - \rho[y_{l_{N_l-N_h+1}} \cdots y_{l_{N_l}}]^T)}{\sigma_l^{-2}\mathbf{1}^\top K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l)^{-1}\mathbf{1} + (1-\rho)^2\sigma_h^{-2}\mathbf{1}^\top K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1}\mathbf{1}},$$

yielding the posterior mean and variance

$$\begin{aligned} m_h(\mathbf{x}^*) &= \beta + \rho K_{\boldsymbol{\theta}_l}(\mathbf{x}^*,\mathbf{X}_l) K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l)^{-1}(\mathbf{y}_l - \beta) + K_{\boldsymbol{\theta}_h}(\mathbf{x}^*,\mathbf{X}_h) K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1}(\mathbf{y}_h - \beta), \\ s_h(\mathbf{x}^*) &= \sigma_h^2 + \rho\sigma_l^2 - \sigma_h^2 K_{\boldsymbol{\theta}_h}(\mathbf{x}^*,\mathbf{X}_h) K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{X}_h)^{-1} K_{\boldsymbol{\theta}_h}(\mathbf{X}_h,\mathbf{x}^*) \\ &\quad - \rho^2\sigma_l^2 K_{\boldsymbol{\theta}_l}(\mathbf{x}^*,\mathbf{X}_l) K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{X}_l)^{-1} K_{\boldsymbol{\theta}_l}(\mathbf{X}_l,\mathbf{x}^*). \end{aligned}$$

### *Appendix B.2. Bayesian Estimation of Model Parameters*

Thanks to the conditional independence assumption in the third line of (8), the first set $(\beta_l, \sigma_l, \boldsymbol{\theta}_l)$ can be separately learned by, for example, the maximum likelihood method using data set $\mathcal{D}_l$, while the parameters of the second Gaussian processes $(\beta_h, \sigma_h, \boldsymbol{\theta}_h)$ and the correlation parameter $\rho$ need to be jointly estimated to carry out the idea of formulation (8)

([12], Section 3.3.2). Similarly to (A1), the parameters of the low-fidelity model are estimated by maximizing the following log-likelihood:

$$-\frac{1}{2}\left[N_l \log(\sigma_l^2) + \log(\det(K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)) + \frac{(\mathbf{y}_l - \beta_l)^\top K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)^{-1}(\mathbf{y}_l - \beta_l)}{\sigma_l^2}\right]. \tag{A5}$$

Differentiating the above equation with respect to $\beta_l$ and $\sigma_l^2$ and solving it at 0 yields

$$\hat{\beta}_l = \frac{\mathbf{1}^\top K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)^{-1}\mathbf{y}_l}{\mathbf{1}^\top K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)^{-1}\mathbf{1}}, \tag{A6}$$

and

$$\hat{\sigma}_l^2 = \frac{1}{N_l}(\mathbf{y}_l - \hat{\beta}_l)^\top K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)^{-1}(\mathbf{y}_l - \hat{\beta}_l). \tag{A7}$$

By substituting (A6) and (A7) into (A5), we have the following optimization problem:

$$\hat{\boldsymbol{\theta}}_l = \arg\min_{\boldsymbol{\theta}_l}\left[N_l \log(\hat{\sigma}_l^2) + \log(\det(K_{\boldsymbol{\theta}_l}(\mathbf{X}_l, \mathbf{X}_l)))\right]. \tag{A8}$$

To learn the second Gaussian process model, we define

$$\mathbf{d} = \mathbf{y}_h - \rho\begin{pmatrix} y_{l_{N_l - N_h + 1}} \\ \vdots \\ y_{l_{N_l}} \end{pmatrix}.$$

The log-likelihood of the model reads

$$-\frac{1}{2}\left[N_h \log(\sigma_h^2) + \log(\det(K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)) + \frac{(\mathbf{d} - \beta_h)^\top K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)^{-1}(\mathbf{d} - \beta_h)}{\sigma_h^2}\right],$$

yielding the MLEs of

$$\hat{\beta}_h = \frac{\mathbf{1}^\top K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)^{-1}\mathbf{d}}{\mathbf{1}^\top K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)^{-1}\mathbf{1}}, \quad \text{and}$$
$$\hat{\sigma}_h^2 = \frac{1}{N_l}(\mathbf{d} - \hat{\beta}_h)^\top K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)^{-1}(\mathbf{d} - \hat{\beta}_h).$$

To find the above parameters, one needs to estimate $\hat{\boldsymbol{\theta}}_h, \rho$ by solving

$$\hat{\boldsymbol{\theta}}_h, \rho = \arg\min_{\boldsymbol{\theta}_h, \rho}\left[N_h \log(\hat{\sigma}_h^2) + \log(\det(K_{\boldsymbol{\theta}_h}(\mathbf{X}_h, \mathbf{X}_h)))\right]. \tag{A9}$$

Note that (A9) can be directly optimized by numerical methods mentioned in Appendix A, as the inversion of the small matrix of size $N_h \times N_h$ is not problematic, while the same procedure may not be applied for system (A8) when the number of low-fidelity $N_l$ is large. In this case, we can use sparse Gaussian processes for the low-fidelity model [34], which reduces the matrix inversion complexity by approximating the full Gaussian process with a set of inducing points, thus improving computational efficiency.

## Appendix C. Sensitivity-Based Pricing Approximation

In order to measure the market risk by Monte Carlo without recalling an expensive pricing engine, many banks use Taylor approximation for portfolio evaluation. This technique is also known as the delta–gamma approximation method in the literature [1,30].

With the same notation used in Section 2, for each scenario of underlying assets $RF_{t+h}$, the corresponding value of the portfolio $V_{t+h} = p_{t+h}(RF_{t+h})$ is approximated by

$$p_{t+h}(RF_{t+h}) \approx p_t(RF_t) + (\delta RF_h)^\top \Delta + \frac{1}{2}(\delta RF_h)^\top \Gamma \, \delta RF_h, \tag{A10}$$

where $\Delta$ and $\Gamma$ are, respectively, the gradient (i.e., delta) and Hessian matrix (i.e., gamma) of the portfolio value $V_t$ with respect to risk factor $RF$, and $\delta RF_h = RF_{t+h} - RF_t$. The estimation of $\Delta$ and $\Gamma$ is similar to the benchmark bump sensitivity approach presented in [36]. Because of the nature of (A10), pioneers in banks usually call this pricing approximation sensitivities times shocks, abbreviated as SxS. In practice, $\Delta$ and $\Gamma$ are not analytically available in closed form, and they are usually estimated by finite difference (or bump and revalue) in practice. If there are $d$ risk factors affecting the portfolio, then one needs to recall the price engine $2d$ times for estimating $\Delta$ by central finite difference ([23], Chapter 8) and additionally $d(d-1)$ times for estimating $\Gamma$. This approximation pricing approach provides a significant computational improvement in risk calculation for small and medium numbers of risk factors $d$, i.e., less than 100. However, its precision can reach an acceptable level once several conditions are met: the portfolio value is well approximated by a linear or quadratic functions of risk factors, the shocked risk factors $RF_{t+h}$ remain locally in the neighborhood of their initial values $RF_t$, and the $\Delta$ and $\Gamma$ must be accurately estimated.

## Appendix D. Neural Network Regression

Alongside Gaussian processes, the neural network is also a powerful nonlinear model for interpolation. Unlike GP regression, the neural network can be learned quickly and is scalable with the number of training points; however, the model requires a sufficient training data set to learn. In this section, we provide the main notation and concept of neural networks, referring the reader to [43] for a detailed presentation.

We denote by $\mathcal{NN}_{d,o,h,u,\sigma}$ a neural network family of $h$ hidden layers, $u$ hidden units, and activation function $\sigma$; furthermore, this family maps input vectors in $\mathcal{X} \in \mathbb{R}^d$ to output vectors in $\mathcal{Y} \in \mathbb{R}^o$. Functions in this family are represented by the following sequential mapping (see also Figure A1):

$$\begin{aligned} \mathbf{a}^0 &= \mathbf{z}^0 = \mathbf{x} \\ \mathbf{a}^l &= \sigma(\mathbf{z}^l) = \sigma\left(\mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l\right) \quad , l = 1, \ldots, h \\ z^{h+1} &= (\mathbf{w}^{h+1})^\top \mathbf{a}^h + b^{h+1}. \end{aligned}$$

This family of functions is parameterized by weights $\mathbf{W}^1 \in \mathbb{R}^{u \times d}, \ldots, \mathbf{W}^l \in \mathbb{R}^{u \times u}, \ldots, \mathbf{w}^{h+1} \in \mathbb{R}^u$ and biases $\mathbf{b}^1 \in \mathbb{R}^u, \ldots \mathbf{b}^l \in \mathbb{R}^u, \ldots, b^{h+1} \in \mathbb{R}^o$. We denote the set of trainable parameters as $\theta = \left(\mathbf{W}^1, \ldots, \mathbf{W}^{h+1}, \mathbf{b}^1, \ldots, \mathbf{b}^h, b^{h+1}\right)$. Additionally, the nonlinear activation function $\sigma$, element-wisely applied, plays a crucial role in establishing the complex mapping between the inputs and outputs of the network. The rectified linear unit function (Relu), i.e., $\mathrm{Relu}(x) = \max(x, 0)$, is usually chosen, which is also our case, because of its generalization property ([43], Section 6.3.1).

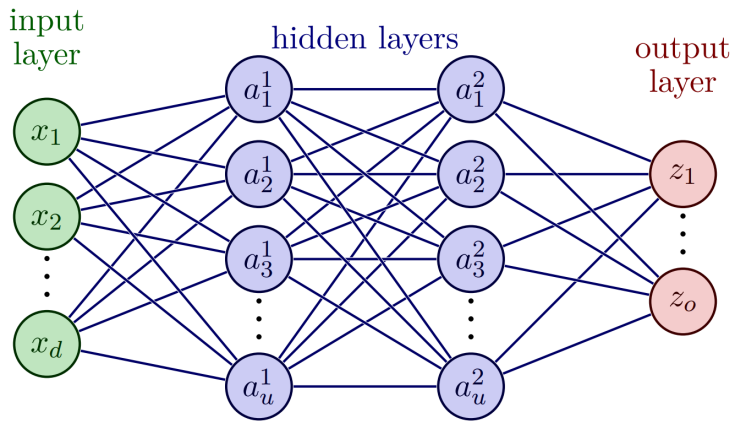


**Figure A1.** Two-hidden-layer neural network architecture.

Given a set of training data $\{(\mathbf{x}_i, y_i)_{i=1\ldots N}\}$, the neural network regression aims at minimizing the following mean square error:

$$\hat{f} \in \underset{f \in \mathcal{NN}_{d,o,h,u,\sigma}}{\arg\min} \frac{1}{N} \sum_{i=1}^{N} \|f(\mathbf{x}_i) - y_i\|_2^2, \tag{A11}$$

where $\|\cdot\|_2$ is the Euclidean norm notation. Equation (A11) is solved by the combination of the backpropagation technique and (stochastic) gradient descent. The details of these algorithms can be found in ([43], Chapter 8).

## References


1. Britten-Jones, M.; Schaefer, S.M. Non-linear value-at-risk. *Rev. Financ.* **1999**, *2*, 161–187. [CrossRef]
2. Dowd, K. *An Introduction to Market Risk Measurement*; John Wiley & Sons: Chichester, UK, 2003.
3. Dowd, K. *Measuring Market Risk*; John Wiley & Sons: Chichester, UK, 2007.
4. Basel Committee on Banking Supervision. Minimum Capital Requirements for Market Risk. 2019. Available online: https://www.bis.org/bcbs/publ/d457.pdf (accessed on 12 May 2025).
5. Ferguson, R.; Green, A. Deeply learning derivatives. *arXiv* **2018**, arXiv:1809.02233.
6. Financial Stability Board. Artificial Intelligence and Machine Learning in Financial Services: Market Developments and Financial Stability Implications. 2017. Available online: https://www.fsb.org/wp-content/uploads/P011117.pdf (accessed on 12 May 2025).
7. Liu, S.; Oosterlee, C.W.; Bohte, S.M. Pricing options and computing implied volatilities using neural networks. *Risks* **2019**, *7*, 16. [CrossRef]
8. Basel Committee on Banking Supervision. Fundamental Review of the Trading Book: A Revised Market Risk Framework. 2013 Available online: https://www.bis.org/publ/bcbs265.pdf (accessed on 12 May 2025).
9. Crépey, S.; Dixon, M. Gaussian process regression for derivative portfolio modeling and application to CVA computations. *J. Comput. Financ.* **2019**, *24*, 1–35.
10. De Spiegeleer, J.; Madan, D.B.; Reyners, S.; Schoutens, W. Machine learning for quantitative finance: Fast derivative pricing, hedging and fitting. *Quant. Financ.* **2018**, *18*, 1635–1643. [CrossRef]
11. Gardner, J.; Pleiss, G.; Weinberger, K.Q.; Bindel, D.; Wilson, A.G. Gpytorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. *Adv. Neural Inf. Process. Syst.* **2018**, *31*, 7576–7586.
12. Le Gratiet, L. Multi-Fidelity Gaussian Process Regression for Computer Experiments. Ph.D. Thesis, Université Paris-Diderot-Paris VII, Paris, France, 2013. Available online: https://theses.hal.science/tel-00866770/PDF/manuscrit.pdf (accessed on 12 May 2025).
13. Lehdili, N.; Oswald, P.; Gueneau, H. Market Risk Assessment of a Trading Book Using Statistical and Machine Learning. 2019. Available online: https://www.researchgate.net/publication/337059465_Market_Risk_Assessment_of_a_trading_book_using_Statistical_and_Machine_Learning?channel=doi&linkId=5dc2cf8da6fdcc21280babf0&showFulltext=true (accessed on 12 May 2025). [CrossRef]
14. Wilkens, S. Machine learning in risk measurement: Gaussian process regression for value-at-risk and expected shortfall. *J. Risk Manag. Financ. Institutions* **2019**, *12*, 374–383. [CrossRef]

15. Ruf, J.; Wang, W. Neural networks for option pricing and hedging: A literature review. *J. Comput. Financ.* **2019**, *24*, 1–46.
16. Meng, X.; Karniadakis, G.E. A composite neural network that learns from multi-fidelity data: Application to function approximation and inverse pde problems. *J. Comput. Phys.* **2020**, *401*, 109020. [CrossRef]
17. Huré, C.; Pham, H.; Warin, X. Deep backward schemes for high-dimensional nonlinear PDEs. *Math. Comput.* **2020**, *89*, 1547–1579. [CrossRef]
18. Goudenège, L.; Molent, A.; Zanette, A. Variance reduction applied to machine learning for pricing Bermudan/American options in high dimension. Oleg Kudryavtsev, Antonino Zanette. In *Applications of Lévy Processes*; Nova Science Publishers: Hauppauge, NY, USA, 2021.
19. Fernández-Godino, M.G.; Park, C.; Kim, N.-H.; Haftka, R.T. Review of multi-fidelity models. *arXiv* **2016**, arXiv:1609.07196.
20. Brevault, L.; Balesdent, M.; Hebbal, A. Overview of gaussian process based multi-fidelity techniques with variable relationship between fidelities, application to aerospace systems. *Aerosp. Sci. Technol.* **2020**, *107*, 106339. [CrossRef]
21. Roncalli, T. *Handbook of Financial Risk Management*; Chapman and Hall/CRC: Boca Raton, FL, USA, 2020.
22. Hong, L.J.; Hu, Z.; Liu, G. Monte carlo methods for value-at-risk and conditional value-at-risk: A review. *ACM Trans. Model. Comput. Simul. (TOMACS)* **2014**, *24*, 1–37. [CrossRef]
23. Crépey, S. *Financial Modeling, A Backward Stochastic Differential Equations Perspective*; Springer Finance Textbook Series; Springer: Berlin/Heidelberg, Germany, 2013.
24. Longstaff, F.A.; Schwartz, E.S. Valuing American options by simulation: A simple least-squares approach. *Rev. Financ. Stud.* **2001**, *14*, 113–147. [CrossRef]
25. Mu, G.; Godina, T.; Maffia, A.; Sun, Y.C. Supervised machine learning with control variates for american option pricing. *Found. Comput. Decis. Sci.* **2018**, *43*, 207–217. [CrossRef]
26. Ruiz, I.; Zeron, M. *Machine Learning for Risk Calculations: A Practitioner's View*; John Wiley & Sons: Hoboken, NJ, USA, 2021.
27. Brigo, D.; Mercurio, F.; Rapisarda, F.; Scotti, R. Approximated moment-matching dynamics for basket-options pricing. *Quant. Financ.* **2003**, *4*, 1–16. [CrossRef]
28. Rasmussen, C.E.; Williams, C.K. *Gaussian Processes for Machine learning*; The MIT Press: Cambridge, MA, USA, 2006.
29. Murphy, K.P. *Machine Learning: A Probabilistic Perspective*; The MIT Press: Cambridge, MA, USA, 2012.
30. Broadie, M.; Du, Y.; Moallemi, C.C. Risk estimation via regression. *Oper. Res.* **2015**, *63*, 1077–1097. [CrossRef]
31. Abbas-Turki, L.; Crépey, S.; Saadeddine, B. Pathwise CVA regressions with oversimulated defaults. *Math. Financ.* **2023**, *33*, 274–307. [CrossRef]
32. Kennedy, M.C.; O'Hagan, A. Predicting the output from a complex computer code when fast approximations are available. *Biometrika* **2000**, *87*, 1–13. [CrossRef]
33. Forrester, A.I.J.; Sóbester, A.; Keane, A.J. Multi-fidelity optimization via surrogate modelling. *R. Soc. A Math. Phys. Eng. Sci.* **2007**, *463*, 3251–3269.
34. Titsias, M. Variational learning of inducing variables in sparse Gaussian processes. *Proc. Mach. Learn. Res.* **2009**, *5*, 567–574.
35. Barone-Adesi, G.; Whaley, R.E. Efficient analytic approximation of american option values. *J. Financ.* **1987**, *42*, 301–320. [CrossRef]
36. Crépey, S.; Li, B.; Nguyen, H.D.; Saadeddine, B. CVA sensitivities, hedging and risk. *arXiv* **2024**, arXiv:2407.18583.
37. Kingma, D.P.; Ba, J. Adam: A method for stochastic optimization. In Proceedings of the 3rd International Conference on Learning Representations, San Diego, CA, USA, 7–9 May 2015.
38. Paleyes, A.; Mahsereci, M.; Lawrence, N.D. Emukit: A python toolkit for decision making under uncertainty. *Proc. Python Sci. Conf.* **2023**, *22*, 68–75.
39. Nocedal, J.; Wright, S.J. *Numerical Optimization*; Springer: New York, NY, USA, 1999.
40. Gardner, J.; Pleiss, G.; Wu, R.; Weinberger, K.; Wilson, A. Product kernel interpolation for scalable Gaussian processes. *Int. Conf. Artif. Intell. And Stat.* **2018**, *84*, 1407–1416.
41. Blei, D.M.; Kucukelbir, A.; McAuliffe, J.D. Variational inference: A review for statisticians. *J. Am. Stat. Assoc.* **2017**, *112*, 859–877. [CrossRef]
42. Hoffman, M.D.; Blei, D.M.; Wang, C.; Paisley, J. Stochastic variational inference. *J. Mach. Learn. Res.* **2013**, *14*, 1303–1347.
43. LeCun, Y.; Bengio, Y.; Hinton, G. Deep learning. *Nature* **2015**, *521*, 436–444. [CrossRef]


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.